

Xiaomi-TabLDM: A Tabular Foundation Model

Technical Report

Xiaomi-TabLDM Team

We introduce Xiaomi-TabLDM, a tabular large data foundation model for classification and regression via in-context learning, which delivers superior prediction accuracy without requiring task-specific fine-tuning. Pretrained exclusively on synthetic data generated from structural causal models (SCMs), our model enables more flexible context utilization and more efficient capacity scaling.

i) A new performance standard. *Strong regression performance across benchmarks:* Xiaomi-TabLDM ranks 1st on OpenML-CTR23 and 2nd on regression across TALENT, TabArena, and BCCO, demonstrating consistently strong regression performance across four complementary benchmark suites. *Favorable performance-efficiency trade-off:* Xiaomi-TabLDM combines strong predictive performance with substantially lower computational cost. For example, on TabArena regression, it achieves the second-highest Elo while using 82% less training time and 68% less prediction time than the top-ranked TabFM.

ii) Large-scale synthetic pretraining. Xiaomi-TabLDM expands the coverage and diversity of synthetic tabular data used for pretraining. We also adopt a three-stage training strategy together with dual-stream feature grouping, lightweight Attention Residual, and sparse Mixture-of-Experts, enabling Xiaomi-TabLDM to learn richer feature interactions and expert specialization across diverse tabular tasks.

iii) Test-time scaling. Xiaomi-TabLDM further extends tabular prediction through test-time compute scaling, where allocating additional computation at inference time consistently improves predictive performance over the base model.

MODEL Xiaomi-TabLDM

DATE September 2026

GITHUB <https://github.com/xiaomi-research/xiaomi-tabldm>

HUGGING FACE <https://huggingface.co/occams/Xiaomi-TabLDM>

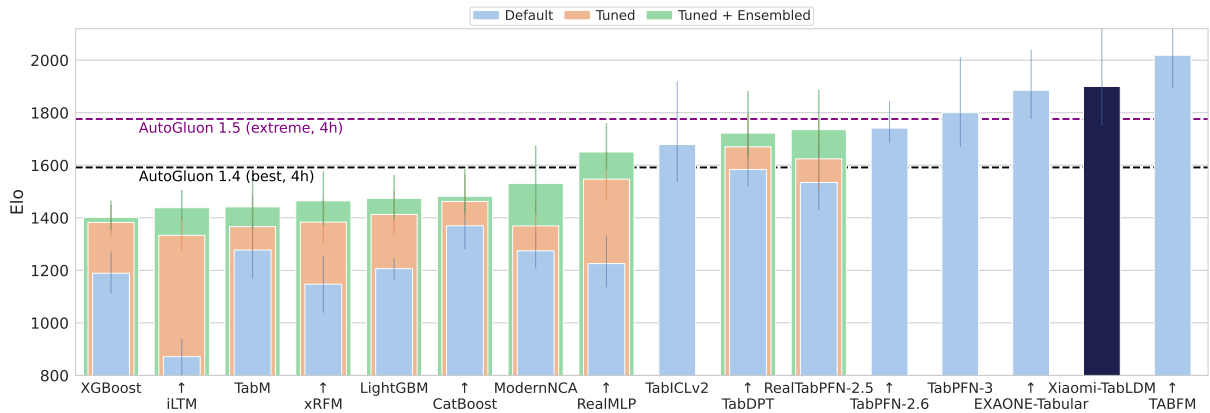


Figure 1. Regression Elo performance on TabArena (higher is better). Xiaomi-TabLDM ranks 2nd among all evaluated methods.

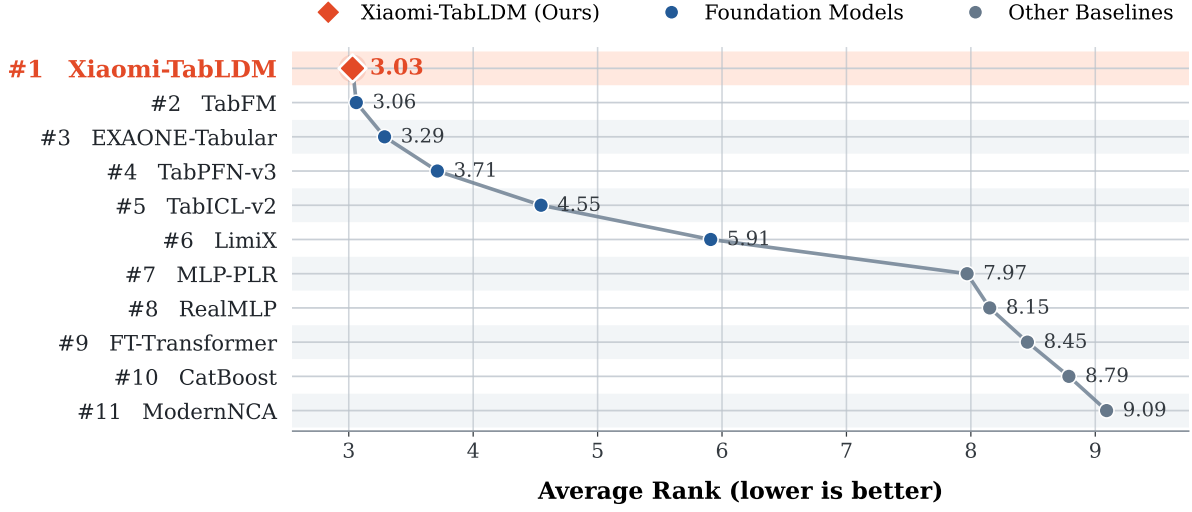


Figure 2. Average-rank comparison on OpenML-CTR23 over 33 regression datasets (lower is better). Xiaomi-TabLDM achieves the best average rank among all evaluated methods.

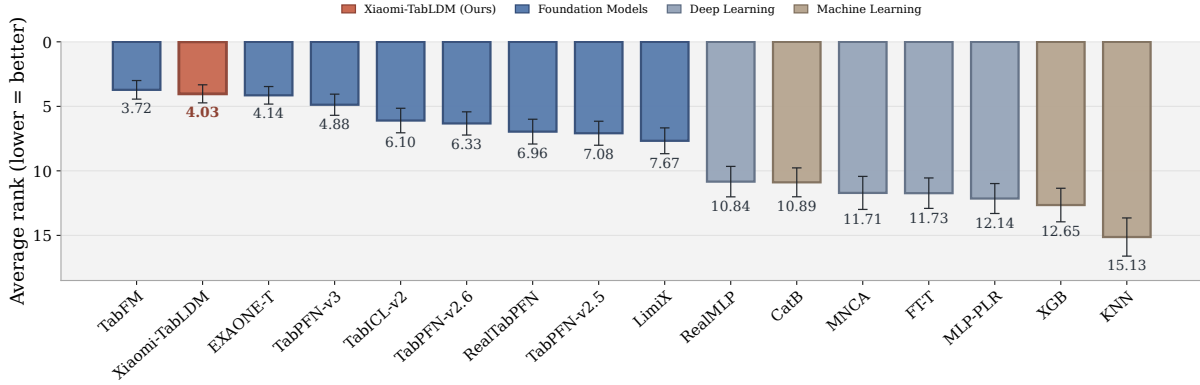


Figure 3. Regression average-rank performance on TALENT (lower is better). Xiaomi-TabLDM ranks 2nd among all evaluated methods.

1 Introduction

Structured data is a foundational modality for scientific analysis and operational decision-making across a wide range of domains, including healthcare, finance, logistics, manufacturing, and public policy [33, 20, 61, 38]. Unlike language or perceptual data, tabular data organizes heterogeneous variables under explicit schemas, preserving numerical scales, missingness patterns, categorical structure, and relationships between features that are essential for reliable prediction and quantitative reasoning. As a result, progress on structured-data modeling remains an important and distinct component of the broader development of general-purpose learning systems.

For decades, prediction on tabular data has been dominated by dataset-specific machine-learning pipelines, particularly gradient-boosted trees and automated ensembles such as XGBoost [10], LightGBM [35], CatBoost [50], and AutoGluon [15]. These methods remain strong and reliable, and extensive empirical studies have shown that boosted trees remain highly competitive with neural networks across diverse tabular problems [43].

Meanwhile, deep learning for tabular data has explored a wide range of architectures, including attention-

based models such as TabNet [2], AutoInt [54], TabTransformer [31], and SAINT [53]; neural decision and feature-interaction models such as NODE [48], DANets [7], DCN-v2 [57], and T2G-Former [59]; and more recent approaches including TANGOS [32], TabCaps [6], TrompT [9], ExcelFormer [8], TabR [24], DNNR [44], and SwitchTab [58]. Despite substantial advances in representation learning, most of these methods still follow the conventional paradigm of training and tuning a separate model for each dataset. Recent surveys have further highlighted the growing interest in adapting large pretrained models and foundation-model paradigms to structured and tabular data [17]. Tabular foundation models [55] provide a different route: a single pretrained model learns across a large distribution of synthetic or real tabular tasks and directly performs prediction on unseen datasets through in-context learning, using labeled examples from the target dataset as context rather than retraining model parameters.

TabPFN established this paradigm by showing that a transformer pretrained over synthetic tabular tasks can perform strong prediction in a single forward pass [28]. Subsequent generations have steadily expanded its practical scope. TabPFN v2 [29] improved robustness to larger datasets and heterogeneous feature types, while TabPFN-2.5 [25] further increased the supported number of rows and features and narrowed the gap with heavily tuned machine-learning ensembles. In parallel, TabICL [51], TabDPT [42], LimiX [63], TabFM [37], EXAONE-Tabular [13] and related approaches have explored larger model capacity, broader training distributions, alternative architectures, and scaling to increasingly diverse tabular settings. Together, these developments have shifted tabular prediction from training a specialized model for each dataset toward building reusable pretrained predictors that transfer across datasets and tasks.

Recent progress in tabular foundation models has increasingly come from jointly scaling training data, model capacity, and inference strategies [26, 52]. Following this direction, we explore a broader synthetic prior, more efficient model scaling, and test-time computation within a unified tabular foundation model. We introduce Xiaomi-TabLDM, a new tabular foundation model that follows and extends this direction. Xiaomi-TabLDM is designed to provide more flexible context utilization while efficiently scaling model capacity. Rather than relying on a single fixed representation pathway, the model learns complementary feature interactions across different granularities and selectively preserves useful information across layers. Sparse expert computation further expands total model capacity while keeping the amount of activated computation limited. Together, these components allow Xiaomi-TabLDM to improve predictive capability without requiring a proportional increase in inference cost.

Across four public benchmark suites, including TALENT [41], TabArena [16], BCCO [63], and OpenML-CTR23 [19], Xiaomi-TabLDM consistently achieves strong performance across diverse classification and regression tasks. Its strongest and most consistent results are observed on regression: Xiaomi-TabLDM ranks first on OpenML-CTR23 and second on regression across TALENT, TabArena, and BCCO. In particular, on OpenML-CTR23, a benchmark dedicated exclusively to tabular regression, Xiaomi-TabLDM achieves the best average rank among all evaluated methods. On TabArena, Xiaomi-TabLDM achieves the second-highest regression Elo, outperforming strong recent tabular foundation models including TabPFN-3, TabPFN-2.6, and TabICL-v2.

Beyond regression, Xiaomi-TabLDM also remains highly competitive across mixed-task evaluations. On TALENT, it ranks second overall and achieves the best performance on binary classification. On TabArena, it ranks fourth overall across 51 datasets and 816 tasks, while on BCCO it maintains top-tier aggregate performance across classification and regression settings.

Beyond predictive performance, Xiaomi-TabLDM provides a favorable performance–efficiency trade-off. On TabArena regression, Xiaomi-TabLDM achieves the second-highest Elo while requiring 82% less training time and 68% less prediction time than TabFM. At a model scale comparable to TabPFN-3, Xiaomi-TabLDM also achieves stronger predictive performance, with particularly pronounced gains on regression. These results show that Xiaomi-TabLDM combines consistently strong regression performance with competitive overall performance and efficient model scaling.

2 Overall Framework

Xiaomi-TabLDM takes a tabular dataset as input and predicts test targets by conditioning on labeled training examples as in-context demonstrations, without task-specific training. Its design centers on three components: large-scale synthetic pretraining, an efficient architecture that combines flexible context utilization with sparse expert specialization, and test-time compute scaling.

2.1 Architecture

The full architecture of Xiaomi-TabLDM is shown in Figure 4. Xiaomi-TabLDM is a transformer-based [56] in-context learning (ICL) model that processes a table through three sequential stages: column-wise feature embedding, row-wise aggregation, and in-context learning prediction.

- **Column-wise feature embedding.** We first organize the input features into a dual-stream feature grouping, which breaks feature symmetries and yields column representations that are more flexible and robust to feature permutation. Each grouped column is then encoded independently by a Set Transformer [39], whose inducing-point attention summarizes it through a small set of learned inducing tokens instead of attending across all rows, producing the final column embeddings.
- **Row-wise feature aggregation.** For each row, the per-column embeddings are prepended with a set of learnable CLS tokens and processed by a stack of self-attention layers. Within these layers, we introduce lightweight Attention Residual (AttnRes) connections in place of the plain additive residual, where the outputs of the preceding residual blocks are adaptively fused to improve gradient flow and representation reuse across depth. Finally, the CLS tokens query the full sequence, and their concatenated states form one fixed-length vector per row.
- **In-context learning prediction.** The row embeddings for the training and test sets are jointly passed to a transformer that performs ICL: training rows attend to one another to model intra-set structure, while test rows attend only to the training rows to form their predictions. The same lightweight AttnRes connections are retained here, and the feed-forward sub-layer of selected layers is replaced with a sparse Mixture-of-Experts, strengthening the model’s ability to adapt to heterogeneous datasets.

In the Column-wise feature embedding and In-context learning prediction stages, attention uses the query-aware scalable softmax (QASSMax)[52], which rescales queries by input length to improve length generalization to large training sets. Further stage-specific technical details of Xiaomi-TabLDM’s design are presented below:

- **Dual-stream feature grouping.** Xiaomi-TabLDM groups each column with two other columns selected by circular shifts over the m feature columns and encodes the triple with a shared linear layer $\text{Linear}(\mathbb{R}^3 \rightarrow \mathbb{R}^d)$. Given an offset set $\mathcal{S} = (\delta_1, \delta_2, \delta_3)$, the embedding of column j is

$$E[i, j] = \text{Linear} \left(x_{i, (j+\delta_1) \bmod m}, x_{i, (j+\delta_2) \bmod m}, x_{i, (j+\delta_3) \bmod m} \right) \quad (1)$$

Xiaomi-TabLDM runs two such streams that differ only in $\mathcal{S}$. The first stream follows TabICLv2[52] with fixed dyadic offsets $(1, 2, 4)$, while the second uses width-adaptive offsets spread geometrically toward a target span s , $(1, \lfloor \sqrt{s} \rfloor, s)$ with $s = \min(s_{\max}, m - 1)$, so it stretches as wide as the table allows without wrapping past it. When the table has few columns, distinct offsets can map to the same column modulo m . Such collisions are resolved by shifting one offset so that the three grouped columns stay distinct and the two streams remain complementary. The resulting embeddings from the two streams are then summed to form the column-wise input embedding.

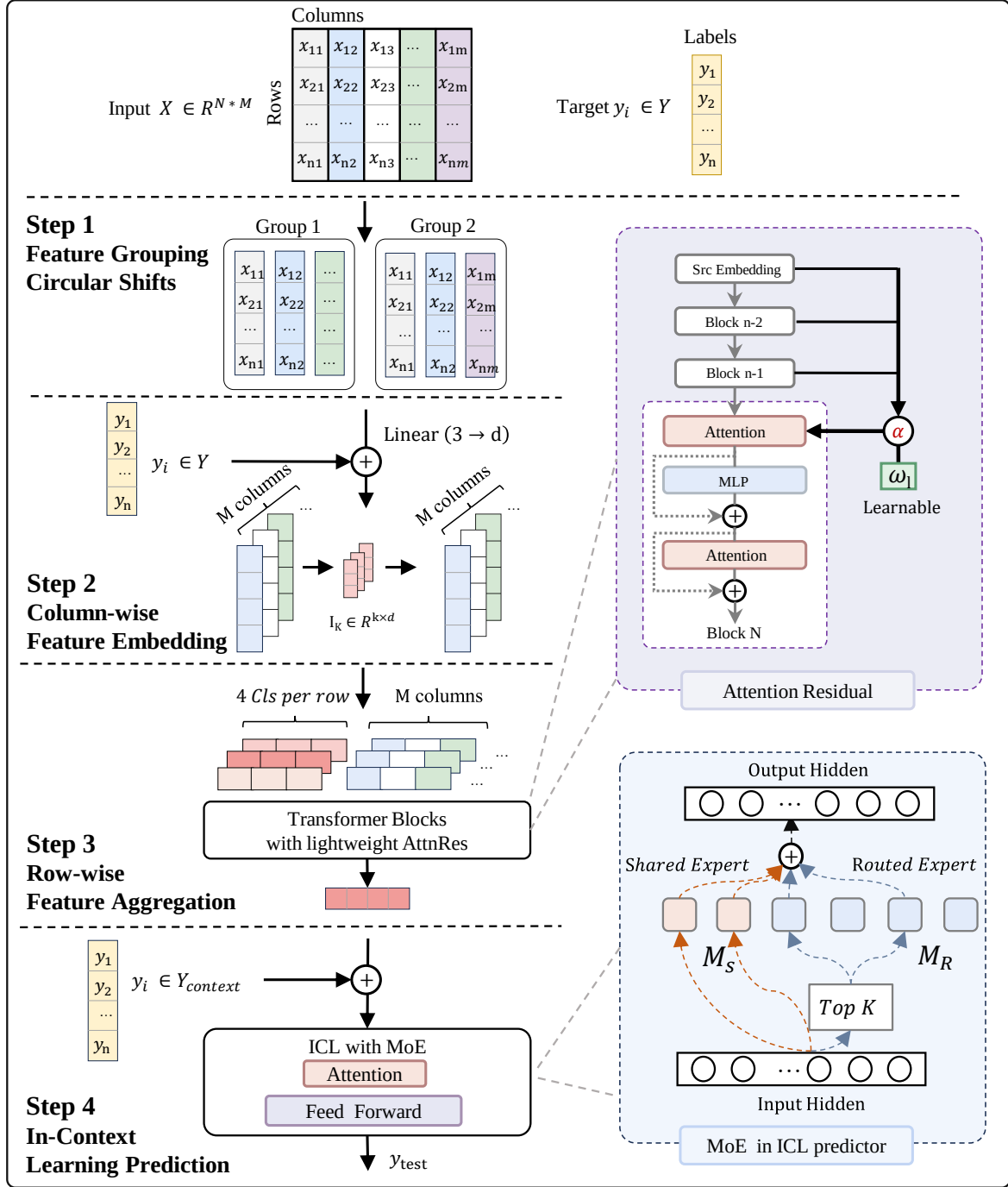


Figure 4. Overall architecture of Xiaomi-TabLDM, which introduces an ICL architecture with a dual-stream feature grouping scheme, lightweight Attention Residual connections, and a sparse Mixture-of-Experts (MoE) design, where a router activates the top- K of M_R routed experts per token together with M_S shared experts.

- **Lightweight AttnRes connections.** We introduce a lightweight inter-layer connection that improves how information propagates across depth, inspired by Block AttnRes [36]. A standard residual connection feeds each layer only the previous output, aggregating all earlier outputs with fixed, uniform weights. The lightweight AttnRes replaces this fixed sum with learned, input-dependent

weights, letting each layer selectively retrieve the earlier representations it needs. For efficiency, the layers are partitioned into blocks and the mechanism is applied only at a fixed stride in depth, so a layer attends over the completed block representations rather than every prior layer. At layer l , a learnable pseudo-query w_l scores each candidate block, and a softmax over depth turns these scores into the aggregation weights:

$$\alpha_j^l = \frac{\exp(w_l^\top \text{RMSNorm}(b_j))}{\sum_{i=0}^{l-1} \exp(w_l^\top \text{RMSNorm}(b_i))} \quad (2)$$

This gives selected layers learned access to both shallow and deep features, improving gradient flow and stabilizing training.

- **Sparse Mixture-of-Experts.** We redesign the ICL predictor by replacing the feed-forward network of selected layers with a sparse Mixture-of-Experts (MoE)[11, 40] that scales capacity for greater representational flexibility. For each row token, a linear router scores the M_R routed experts and a top- K operator activates the K highest-scoring ones, whose outputs are combined with those of M_S shared experts applied to every token. Since only K experts are activated per token, the parameter count scales with M_R while the per-token cost stays proportional to K feed-forward layers, independent of M_R . The shared experts learn the common transformation reused across all tokens, so that the routed experts, rather than redundantly relearning it, can devote their capacity to specialization.

2.2 Pretraining

We pretrain classification and regression models separately on synthetically generated tabular datasets, using a three-stage curriculum that progressively scales the dataset size. The classifier is trained with a cross-entropy loss and the regressor with a pinball loss over 999 quantiles.

- **Stage 1.** 500K steps for classification and 300K for regression, on datasets of 1,024 samples with 30–90% used for training, a maximum learning rate of 8e-4, and gradient clipping at 10. This stage uses the default feed-forward blocks. Enabling AttnRes stabilizes convergence and accounts for the shorter regression schedule; without it, the model collapses at an earlier step on large datasets (roughly beyond 1,200 samples).
- **Stage 2.** 40K steps on datasets of 400–10,240 samples (log-uniform), $\sim 80\%$ used for training, a maximum learning rate of 1e-4, and gradient clipping at 10. MoE and its auxiliary losses are enabled from this stage on.
- **Stage 3.** 10K steps on datasets of 400–60,000 samples (log-uniform), $\sim 80\%$ used for training, a maximum learning rate of 2e-5, and gradient clipping at 1, adapting the model to long-context, large-sample inference.

Optimizer and pre-training cost. We train with the Muon optimizer[34] based on the implementation of TabICLv2[52], together with a cosine learning-rate schedule and a weight decay of 0.01. Pre-training runs on 8 A100 GPUs (80 GB): around 7–10 days for **Stage 1**, and 2 days each for **Stage 2** and **Stage 3**, with FlashAttention-2[12] enabled from **Stage 2** onward.

2.3 Synthetic Prior

Following previous tabular foundation models [28, 51], Xiaomi-TabLDM is pretrained entirely on synthetic datasets generated from a structural causal model (SCM) prior [46, 47]. The prior is designed to maximize

the diversity of dataset structures, variable types, and functional relationships while retaining learnable predictive signals [64]. Figure 5 summarizes the complete generation pipeline.

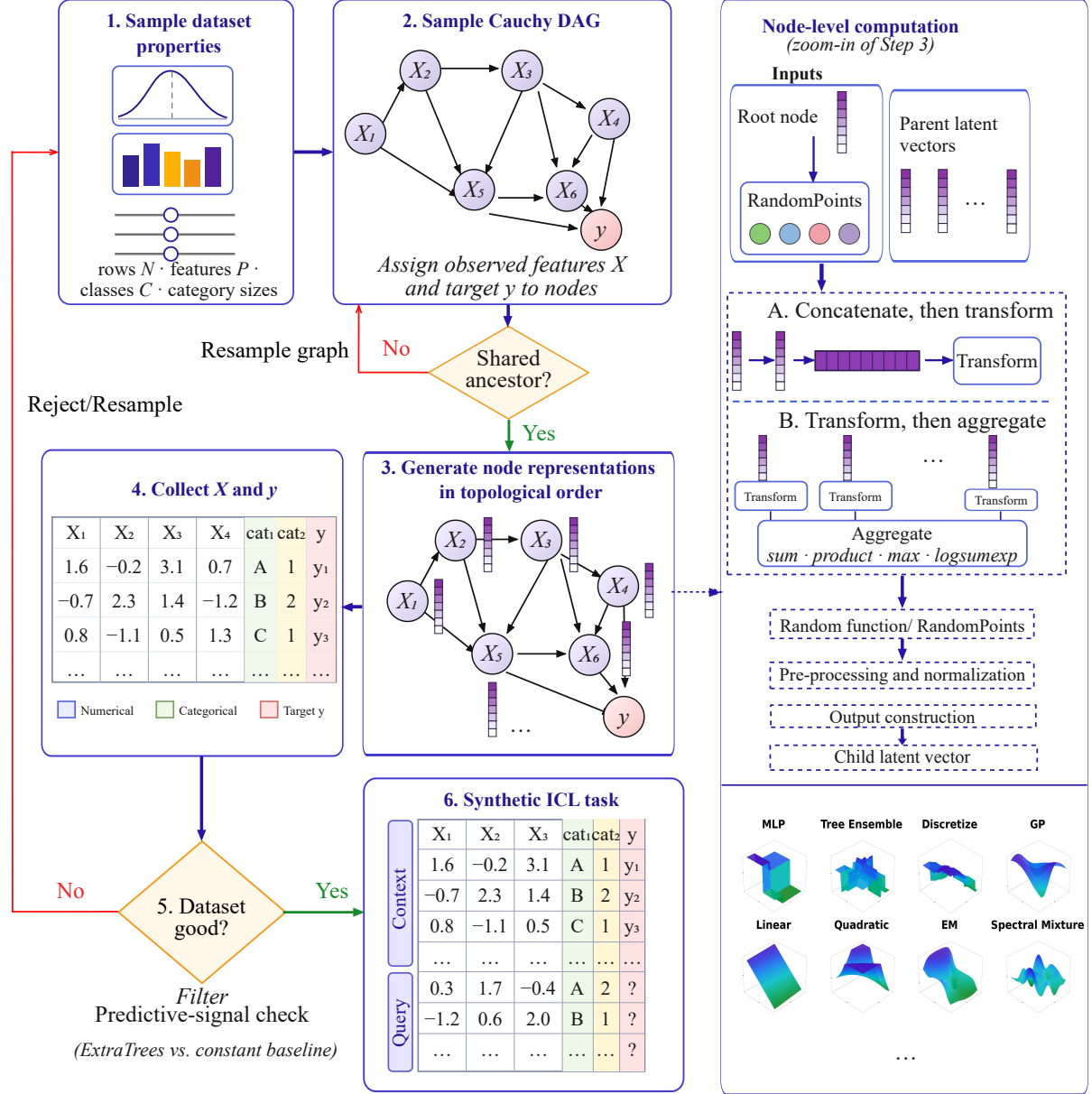


Figure 5. Schematic overview of the synthetic data generation prior. The dataset-level pipeline samples dataset properties and a Cauchy DAG, checks whether the selected features and target share an ancestor, generates node representations in topological order, and collects the resulting tabular data. Datasets passing the predictive-signal check are split into context and query sets to form synthetic in-context learning tasks. The right panel details the node-level computation and the 12 random-function classes used by the prior.

We organize its main components into the following four stages:

- 1. Dataset configuration.** We first sample the global properties of each task, including its size, numerical and categorical feature counts, categorical cardinalities, target type, and context–query split.

2. **Graph-based data generation.** We sample a directed acyclic graph using the random Cauchy graph mechanism of TabICL-v2 [52]. Latent representations are propagated through the graph in topological order using the node computations illustrated in the middle row of Figure 5, with additional Gaussian noise injected to increase data diversity and robustness. We also add additional transformation functions to broaden the range of smoothness, sparsity, additivity, and local structure represented by the prior.
3. **Tabular data construction.** We assign the observed features and target to sampled graph nodes and collect their values after evaluating the graph. Numerical features and regression targets are obtained from continuous latent dimensions, whereas categorical features and classification targets are produced through discretization or categorical sampling. As illustrated in Figure 5, only selected dimensions are observed. The remaining dimensions introduce latent variation. Random scaling further varies the relative importance of features and graph nodes across tasks.
4. **Postprocessing and quality filtering.** We standardize and randomly permute the resulting tables before removing invalid or uninformative tasks. A graph is resampled when the features and target share no common ancestor. We also reject tasks for which an ExtraTrees model [22] cannot reliably outperform a constant predictor. Finally, we divide each retained dataset into context and query samples for in-context pretraining.

Together, these stages generate diverse classification and regression tasks by varying dataset configurations, dependency structures, functional relationships, and observed feature types.

3 Test-Time Scaling

Test-time scaling improves predictive performance by allocating additional inference-time computation while keeping the pretrained model fixed. Recent tabular foundation models have implemented this idea by aggregating predictions across multiple estimators, dataset permutations, and feature transformations [26, 52]. TabFM further combines predictions generated from SVD and cross-feature representations using nonnegative least squares [37]. Building on these approaches, we introduce a context-adaptive framework that enhances inference based on the given context. This allows the inference pipeline to adapt to heterogeneous datasets without updating the pretrained model.

Our test-time scaling strategy consists of the following components:

1. **Feature shuffling and sampling.** Each ensemble member uses a randomly shuffled feature order and randomly samples a subset of features for training and inference. This reduces sensitivity to column ordering and specific feature combinations, creates prediction diversity at negligible additional cost, and enhances robustness in high-dimensional scenarios.
2. **Diverse preprocessing views.** We construct complementary views through different normalization schemes, quantile transformation, SVD-based representations, and feature interactions. For regression, we additionally consider target transformations for heavy-tailed outcomes.
3. **NNLS-based combination.** For both classification and regression, we learn nonnegative weights for the estimators corresponding to the retained inference modes. For large datasets, we use predictions on this single holdout set to estimate the weights. Otherwise, we generate out-of-fold predictions through K -fold cross-validation to obtain a larger and more stable sample for weight estimation. We then fit nonnegative least squares to one-hot labels for classification or continuous targets for regression and use the resulting weights to combine the candidate inference views.

4. **Final classification adjustment.** The combined class probabilities can be calibrated on validation data to improve probabilistic prediction quality. Final class labels are then obtained by taking the class with the largest adjusted probability.

Overall, the procedure scales inference by increasing the diversity and number of evaluated prediction paths, rather than by modifying the underlying model. The NNLS-based combination step effectively exploits complementary predictions among the diverse inference views, allowing the method to flexibly adapt to heterogeneous datasets. Thus, the approach uses additional test-time computation selectively, harnessing the benefits of ensemble diversity while mitigating the risk of overfitting through the nonnegative constraint on combination weights.

4 Evaluation

In this section, we evaluate Xiaomi-TabLDM on four public tabular learning benchmarks, TALENT [41], TabArena [16], BCCO [63], and OpenML-CTR23 [19], covering a broad range of real-world classification and regression tasks.

Table 1 summarizes the benchmark collections used in our evaluation, covering four benchmark suites with complementary task types and dataset scales. TALENT is the largest collection, containing 300 datasets, including 200 classification and 100 regression tasks, while BCCO contains 156 datasets with a similar mixture of classification and regression problems. TabArena provides 51 datasets spanning both task types. In addition, OpenML-CTR23 focuses exclusively on regression. Across these benchmarks, most datasets contain fewer than 10K training examples, with a smaller portion ranging from 10K to 100K; TALENT additionally includes several datasets with more than 100K training instances.

In Section 4.1, we evaluate Xiaomi-TabLDM on TALENT, reporting regression, overall, and binary-classification performance. In Section 4.2, we report the overall and regression results on TabArena and further analyze performance–efficiency trade-offs and pairwise model comparisons. We further evaluate Xiaomi-TabLDM on BCCO and OpenML-CTR23 in Sections 4.3 and 4.4, respectively.

Table 1. Statistics of the benchmark suites considered in our evaluation. We report the total number of datasets, task-type composition, classification-task breakdown, and dataset-size distribution for each benchmark collection.

Benchmark	# Datasets	Task Type		Classification Type		Training Set Size		
		Cls.	Reg.	Binary	Multiclass	< 10K	10K–100K	> 100K
TALENT	300	200	100	120	80	218	78	4
BCCO	156	106	50	71	35	128	28	-
TabArena	51	38	13	30	8	36	15	-
OpenML-CTR23	33	-	33	-	-	23	10	-

4.1 TALENT

TALENT [41] is a unified benchmark and toolbox for evaluating tabular learning methods under consistent preprocessing, hyperparameter tuning, and evaluation protocols. Its benchmark suite contains 300 datasets spanning 120 binary classification, 80 multiclass classification, and 100 regression tasks, covering diverse dataset sizes and application domains. TALENT includes a broad range of classical machine-learning, tree-based, deep tabular, and recent foundation-model approaches, providing a complementary evaluation setting to TabArena for assessing model performance across different types of tabular prediction tasks.

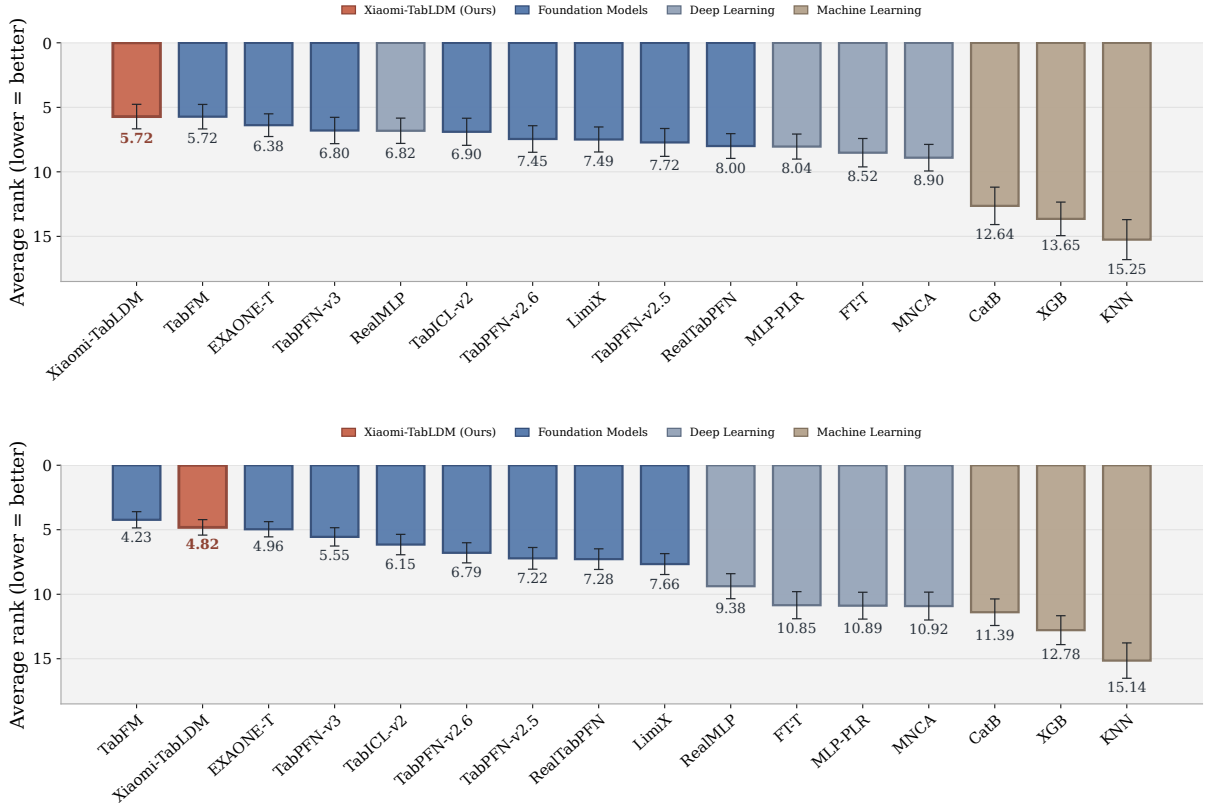


Figure 6. Performance on the TALENT benchmark following the TabICLv2 evaluation protocol [52]. For methods with missing results on a subset of datasets, the corresponding score entries are imputed using K-nearest-neighbour values following the evaluation protocol. The top panel reports binary-classification performance, while the bottom panel reports overall performance across task types. Results are measured by average rank across datasets (lower is better); error bars indicate 95% bootstrap confidence intervals.

4.1.1 Main Results

Regression performance. Xiaomi-TabLDM performs particularly strongly on regression, achieving an average rank of **4.03** across 100 regression datasets and ranking **second** among all evaluated methods, behind only TabFM (3.72). It outperforms other strong tabular foundation models, including EXAONE-Tabular (4.15), TabPFN-v3 (4.88), TabICL-v2 (6.10), and TabPFN-v2.6 (6.33). Together with its strong results on other regression benchmarks, this demonstrates the consistent regression capability of Xiaomi-TabLDM across diverse tabular datasets.

Overall performance. As shown in Figure 6 (bottom), Xiaomi-TabLDM achieves an average rank of **4.82**, ranking **second overall**, behind only TabFM (4.23). It outperforms other recent tabular foundation models, including EXAONE-Tabular (4.96), TabPFN-v3 (5.55), TabICL-v2 (6.15), and TabPFN-v2.6 (6.79). These results show that the strong regression performance of Xiaomi-TabLDM extends to competitive performance across the broader range of tabular tasks covered by TALENT.

Classification. As shown in Figure 6 (top), Xiaomi-TabLDM also performs strongly on binary classification, achieving an average rank of **5.72** and tying with TabFM for the best performance among all evaluated methods. It further outperforms EXAONE-Tabular (6.38), TabPFN-v3 (6.80), and TabICL-v2 (6.90).

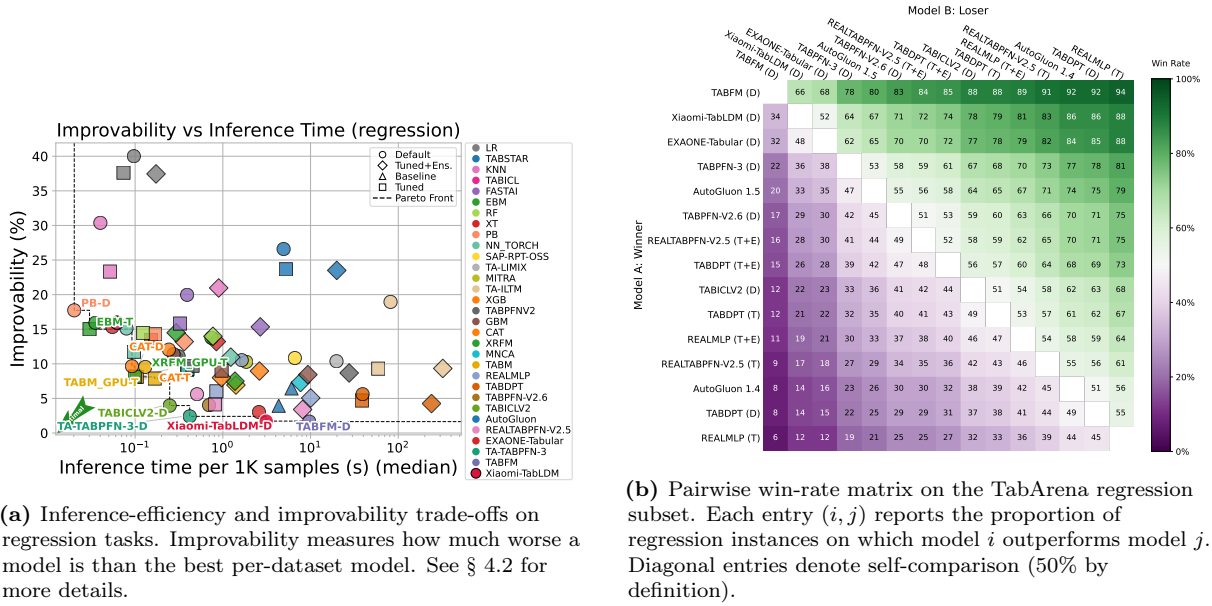


Figure 7. Regression task performance analysis: (left) efficiency-improvability trade-offs; (right) pairwise model comparisons.

4.2 TabArena

We also evaluate Xiaomi-TabLDM on TabArena [16], consisting of 51 datasets and 816 complete tasks, including 38 classification datasets and 13 regression datasets. We use Elo as the primary evaluation metric and additionally report the number of dataset wins, improvability, and training and prediction time. We report both overall and regression performance, and further analyze performance–efficiency trade-offs and pairwise model comparisons.

Our primary comparisons include leading gradient-boosted tree frameworks, including XGBoost [10], CatBoost [50], and LightGBM [35], as well as strong recent tabular foundation models, including TabICLv2 [52], TabPFN [28], TabPFN-3 [26], RealTabPFN [21], and TabFM [37]. We also compare against AutoGluon [15] as a strong AutoML baseline.

4.2.1 Main Results

Regression. On the 13 regression datasets, as shown in Table 2, Xiaomi-TabLDM achieves an Elo score of **1900**, ranking **second** among all evaluated methods by Elo point estimate, behind only TabFM (2019). It outperforms other strong tabular foundation models, including EXAONE-Tabular (1885), TabPFN-3 (1800), TabPFN-2.6 (1741), and TabICLv2 (1679), as well as the heavily tuned AutoGluon 1.5 extreme configuration (1776).

Beyond Elo, Xiaomi-TabLDM achieves **1.8 dataset wins** and the **second-lowest improvability of 1.7%**, only marginally behind TabFM (1.6%). The pairwise win-rate matrix in Figure 7(b) provides a complementary view: Xiaomi-TabLDM wins **67%** of pairwise comparisons against AutoGluon 1.5 and **78%** against TabICLv2. Together, these results demonstrate strong and consistent regression performance across the TabArena benchmark.

Table 2. Performance on the regression subset of TabArena, covering 13 datasets. We compare models in terms of Elo, number of wins, improvability, and training and prediction time. Here **D** denotes the default (untuned) model, **T** the fine-tuned model, and **T+E** ensembling after fine-tuning. Xiaomi-TabLDM achieves the second-highest Elo while maintaining substantially lower computational cost than most tuned and ensembled baselines.

Model	Elo ($\uparrow$)	#wins ($\uparrow$)	Improvability ($\downarrow$)	Train time per 1K [s]	Predict time per 1K [s]
TABFM (D)	2019 _{-127,+181}	3.7	1.6%	38.85	9.67
Xiaomi-TabLDM (D)	1900 _{-148,+278}	1.8	1.7%	6.99	3.12
EXAONE-Tabular (D)	1885 _{-110,+155}	1.5	3.0%	13.71	2.58
TabPFN-3 (D)	1800 _{-131,+211}	0.9	2.4%	3.87	0.42
AutoGluon 1.5 (extreme, 4h)	1776 _{-96,+137}	1.3	3.9%	335.03	4.33
TabPFN-2.6 (D)	1741 _{-56,+104}	0.1	4.0%	8.52	0.70
RealTabPFN-2.5 (T+E)	1736 _{-103,+153}	0.2	3.4%	1709.05	8.12
TabDPT (T+E)	1722 _{-90,+160}	1.5	4.3%	4786.60	239.30
TabICLv2 (D)	1679 _{-142,+242}	0.5	4.0%	2.10	0.25
TabDPT (T)	1670 _{-73,+128}	0.0	4.7%	4786.60	38.50
RealMLP (T+E)	1650 _{-67,+111}	0.1	5.1%	3995.01	10.05
RealTabPFN-2.5 (T)	1624 _{-117,+154}	0.2	4.1%	1709.05	0.81
AutoGluon 1.4 (best, 4h)	1592 _{-90,+117}	0.0	6.4%	1866.35	6.07
TabDPT (D)	1584 _{-64,+137}	0.0	5.6%	46.62	39.21
RealMLP (T)	1547 _{-82,+105}	0.0	6.0%	3995.01	0.84
RealTabPFN-2.5 (D)	1534 _{-106,+141}	0.0	5.6%	7.04	0.51
ModernNCA (T+E)	1531 _{-118,+145}	0.6	7.3%	3779.70	7.69
CatBoost (T+E)	1482 _{-65,+103}	0.0	8.0%	3555.27	0.96
LightGBM (T+E)	1474 _{-83,+89}	0.0	8.4%	700.19	9.32
xRFM (T+E)	1464 _{-95,+112}	0.0	7.5%	714.50	1.38
TabM (T+E)	1442 _{-85,+130}	0.0	6.9%	4160.58	1.41
XGBoost (T+E)	1402 _{-47,+64}	0.0	9.0%	834.93	2.61
CatBoost (D)	1369 _{-89,+93}	0.0	9.7%	10.89	0.09
ModernNCA (D)	1274 _{-68,+81}	0.0	10.8%	15.50	0.30
LimiX (D)	1246 _{-154,+167}	0.1	10.4%	74.68	19.76
RealMLP (D)	1226 _{-91,+107}	0.0	10.5%	8.90	1.64
LightGBM (D)	1206 _{-41,+41}	0.0	11.4%	2.11	0.27
XGBoost (D)	1189 _{-77,+83}	0.0	12.0%	2.24	0.24
TabPFNv2 (D)	1184 _{-143,+133}	0.0	11.1%	2.80	0.31
RandomForest (T+E)	1162 _{-61,+62}	0.0	14.0%	515.75	0.77
xRFM (D)	1147 _{-109,+110}	0.0	13.8%	2.45	0.74
ExtraTrees (D)	1069 _{-107,+94}	0.0	15.3%	0.47	0.06
RandomForest (D)	1000 _{-71,+44}	0.0	15.9%	0.53	0.06
FastaiMLP (D)	864 _{-160,+111}	0.0	20.0%	2.60	0.39
KNN (D)	680 _{-246,+170}	0.0	30.4%	0.19	0.04
Linear (D)	295 _{-413,+146}	0.0	40.0%	0.95	0.10

Overall Performance. Across all 51 datasets and 816 tasks, as shown in Table 11, Xiaomi-TabLDM achieves an Elo score of **1659**, ranking **fourth** by Elo point estimate. Its performance is nearly tied with AutoGluon 1.5 extreme (1662) and surpasses TabPFN-3 (1650), TabPFN-2.6 (1596), RealTabPFN-2.5 with tuning and ensembling (1579), and TabICLv2 (1576). Moreover, Xiaomi-TabLDM achieves **3.1 dataset wins**, exceeding TabPFN-3 and other recent tabular foundation models except TabFM and EXAONE-Tabular.

The overall pairwise win-rate matrix in Figure 11 provides an additional view of its relative performance across the full benchmark. Together with its second-place result on regression, these results show that

Xiaomi-TabLDM remains highly competitive across diverse TabArena tasks, with regression emerging as its strongest regime.

4.2.2 Performance–Efficiency Trade-offs

Figure 7(a) further examines the trade-off between regression performance and inference efficiency. Improvability (y-axis, lower is better) measures the relative performance gap to the best-performing method on each dataset, while the x-axis reports median inference time per 1K samples.

Xiaomi-TabLDM achieves an improvability of only **1.7%** with an inference time of **3.12 s** per 1K samples, placing it in the high-performance region of the trade-off space. Compared with TabFM, which achieves a slightly lower improvability of 1.6%, Xiaomi-TabLDM requires **68% less prediction time** (3.12 s vs. 9.67 s). Table 2 further shows that Xiaomi-TabLDM requires **82% less training time** (6.99 s vs. 38.85 s).

Compared with TabPFN-3 at a similar model scale, Xiaomi-TabLDM also achieves stronger regression performance, improving Elo from 1800 to 1900 and reducing improvability from 2.4% to 1.7%. Overall, these results show that Xiaomi-TabLDM achieves a favorable performance–efficiency trade-off, approaching the strongest regression performance on TabArena while requiring substantially less computation than TabFM.

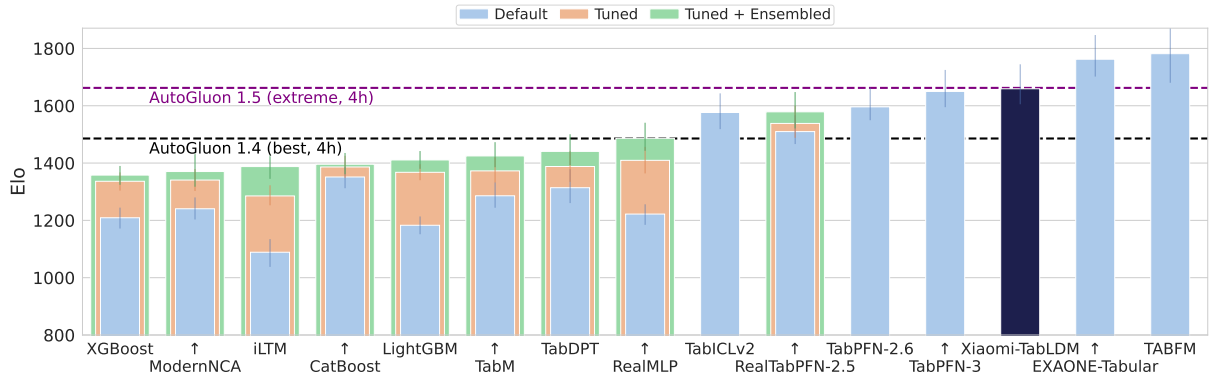


Figure 8. Overall Elo performance on TabArena (higher is better).

4.3 BCCO

We further evaluate Xiaomi-TabLDM on the Balanced Comprehensive Challenging Omni-domain (BCCO) benchmark [63], which consists of two subsets, BCCO-CLS for classification and BCCO-REG for regression. BCCO is constructed from a broad collection of open-source structured datasets after extensive deduplication and cleaning, and is designed to cover heterogeneous real-world prediction settings. Compared with commonly used tabular benchmarks, BCCO contains greater variation in dataset characteristics, including sample size, feature dimensionality, number of classes, categorical-to-numerical feature ratio, sample-to-feature ratio, and missing-value rate. Notably, nearly one-third of the datasets contain missing values, making the benchmark particularly challenging for models that need to generalize across diverse data conditions.

In our evaluation, BCCO-CLS contains 106 classification datasets and BCCO-REG contains 50 regression datasets. The benchmark excludes extremely large datasets with more than 50,000 training samples, 10,000 features, or 10 target classes. Its broad coverage across dataset characteristics enables evaluation over a diverse range of task regimes rather than concentrating on a small subset of dataset types.

Table 3. BCCO mean-rank performance across regression, multiclass classification, and overall evaluations. Lower values are better. Numbers in parentheses denote the ranking position under each metric.

Model	Reg			Multi-Cls			Overall		
	Accuracy ↓	AUC ↓	LogLoss ↓	Accuracy ↓	AUC ↓	LogLoss ↓	Accuracy ↓	AUC ↓	LogLoss ↓
TabFM	2.680 (1)	2.680 (1)	2.680 (1)	2.871 (1)	2.571 (1)	2.286 (1)	3.694 (1)	3.095 (1)	3.064 (1)
Xiaomi-TabLDM	<u>2.940 (2)</u>	<u>2.940 (2)</u>	<u>2.940 (2)</u>	<u>4.171 (2)</u>	<u>4.257 (4)</u>	<u>3.971 (2)</u>	4.451 (3)	4.154 (3)	3.819 (3)
EXAONE-Tabular	3.400 (3)	3.400 (3)	3.400 (3)	4.757 (4)	4.186 (3)	4.086 (4)	<u>4.436 (2)</u>	<u>3.731 (2)</u>	<u>3.625 (2)</u>
TabPFN-v3	3.560 (4)	3.560 (4)	3.560 (4)	5.129 (5)	4.343 (5)	4.629 (5)	4.999 (4)	4.521 (4)	4.407 (4)
TabICL-v2	4.780 (5)	4.780 (5)	4.780 (5)	4.329 (3)	<u>4.114 (2)</u>	4.029 (3)	5.336 (5)	4.805 (5)	4.744 (5)
LimiX	6.500 (6)	6.500 (6)	6.500 (6)	5.143 (6)	5.514 (6)	5.057 (6)	6.321 (6)	6.071 (6)	5.915 (6)
RealMLP	8.330 (7)	8.330 (7)	8.330 (7)	9.614 (10)	10.029 (12)	9.714 (11)	7.229 (7)	8.623 (8)	8.481 (8)
CatBoost	8.960 (8)	8.960 (8)	8.960 (8)	8.800 (8)	9.000 (9)	8.229 (7)	9.400 (11)	8.591 (7)	8.249 (7)
FT-Transformer	9.030 (9)	9.030 (9)	9.030 (9)	9.257 (9)	8.943 (8)	8.886 (10)	8.121 (8)	8.815 (9)	8.555 (9)
MLP-PLR	9.640 (10)	9.640 (10)	9.640 (10)	9.700 (11)	9.986 (11)	10.171 (12)	8.346 (10)	9.414 (11)	9.509 (11)
ModernNCA	9.840 (11)	9.840 (11)	9.840 (11)	8.657 (7)	8.343 (7)	8.657 (8)	8.220 (9)	9.109 (10)	9.245 (10)
LightGBM	10.920 (12)	10.920 (12)	10.920 (12)	10.400 (13)	10.971 (13)	12.543 (13)	10.602 (12)	10.646 (13)	11.918 (13)
XGBoost	11.120 (13)	11.120 (13)	11.120 (13)	10.229 (12)	9.657 (10)	8.800 (9)	10.933 (13)	10.400 (12)	9.961 (12)
KNN	13.300 (14)	13.300 (14)	13.300 (14)	11.943 (14)	13.086 (14)	13.943 (14)	12.914 (14)	13.027 (14)	13.508 (14)

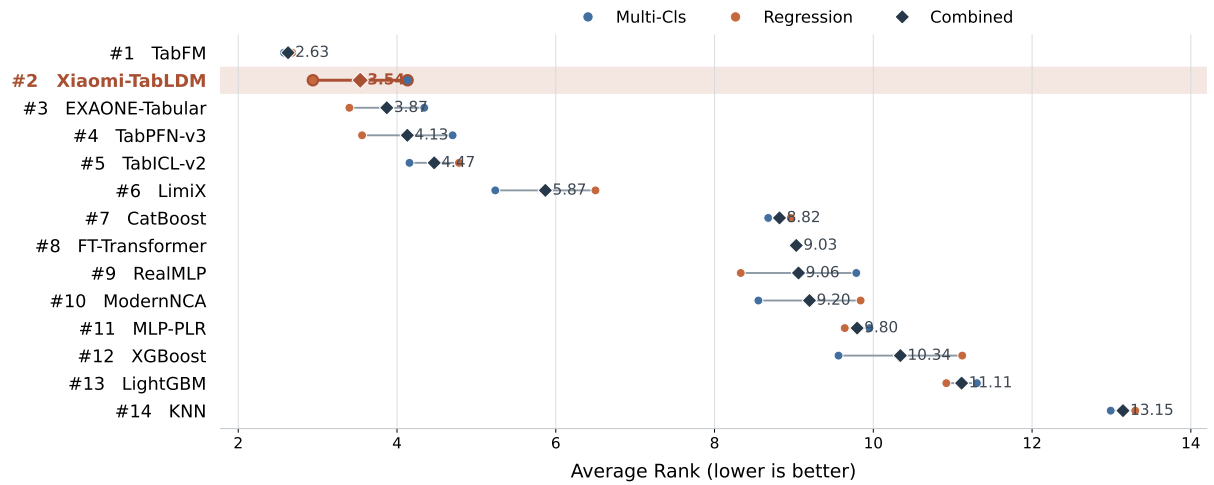


Figure 9. Average-rank comparison on the BCCO benchmark. Circles denote the average ranks on BCCO-CLS and BCCO-REG, while diamonds denote the combined average rank across the two settings. Models are ordered by the combined average rank; lower is better.

Main Results. Table 3 reports the mean-rank performance on BCCO across regression, multiclass classification, and the overall evaluation. Xiaomi-TabLDM demonstrates particularly strong performance on regression, ranking second under all three metrics, with a mean rank of 2.940 for Accuracy, AUC, and LogLoss. It consistently outperforms other strong tabular foundation models, including EXAONE-Tabular, TabPFN-v3, TabICL-v2, and LimiX, and is surpassed only by TabFM.

On multiclass classification, Xiaomi-TabLDM remains highly competitive, ranking second under both

Accuracy (4.171) and LogLoss (3.971), while achieving fourth place under AUC (4.257). When aggregating across the full BCCO benchmark, Xiaomi-TabLDM consistently ranks third under Accuracy (4.451), AUC (4.154), and LogLoss (3.819). Overall, these results show that Xiaomi-TabLDM achieves robust top-tier performance across heterogeneous BCCO tasks, with a particularly clear advantage on regression while maintaining competitive performance on multiclass classification.

4.4 OpenML-CTR23

We further evaluate Xiaomi-TabLDM on the OpenML Curated Tabular Regression benchmarking suite 2023 (OpenML-CTR23), a benchmark specifically designed for tabular regression. The original suite contains 35 regression problems selected according to a set of strict curation criteria, providing a standardized and reliable testbed for comparing regression methods across diverse tabular datasets. In our experiments, we report results on 33 datasets and compare Xiaomi-TabLDM against both tabular foundation models and conventional learning baselines.

Main Results. Figure 2 reports the average-rank results on OpenML-CTR23 [19]. Xiaomi-TabLDM achieves the best average rank of **3.03**, ranking first among all evaluated methods. It also consistently ranks ahead of other strong tabular foundation models, including TabPFN-v3 (3.71), TabICL-v2 (4.55), and LimiX (5.91). These results demonstrate the strong regression capability of Xiaomi-TabLDM and its ability to generalize consistently across the diverse tasks covered by OpenML-CTR23.

4.5 Embeddings.

Finally, we examine whether Xiaomi-TabLDM learns meaningful representations of tabular samples. We extract the row embeddings produced by Xiaomi-TabLDM and apply PCA to project them into two dimensions for visualization. Figure 10 compares PCA applied directly to the original input features with PCA applied to the learned embeddings across three representative datasets. While the original feature space exhibits relatively diffuse and weakly organized structures, the embeddings learned by Xiaomi-TabLDM reveal substantially clearer low-dimensional organization, including coherent curved manifolds and more compact sample structures. This suggests that Xiaomi-TabLDM transforms heterogeneous tabular inputs into representations that better capture the underlying structure of the data.

As shown in Figure 10, Xiaomi-TabLDM learns structured row embeddings from tabular data. The upper plots show 2D PCA applied directly to the original features of three datasets, while the lower plots show PCA applied to the corresponding row embeddings produced by Xiaomi-TabLDM. The learned representations exhibit substantially clearer and more coherent low-dimensional structures than the original feature space.

5 Conclusion

In this report, we present Xiaomi-TabLDM, a tabular foundation model designed to achieve strong predictive performance while maintaining an efficient model and inference profile. Across four public benchmark suites covering diverse classification and regression tasks, Xiaomi-TabLDM consistently ranks among the strongest evaluated methods. Its performance is particularly strong on regression, where it achieves top-tier results across TALENT, TabArena, BCCO, and OpenML-CTR23, while remaining competitive across classification settings.

Beyond predictive accuracy, our evaluation also highlights a favorable performance–efficiency trade-off. At a model scale comparable to TabPFN-3, Xiaomi-TabLDM achieves stronger performance in several

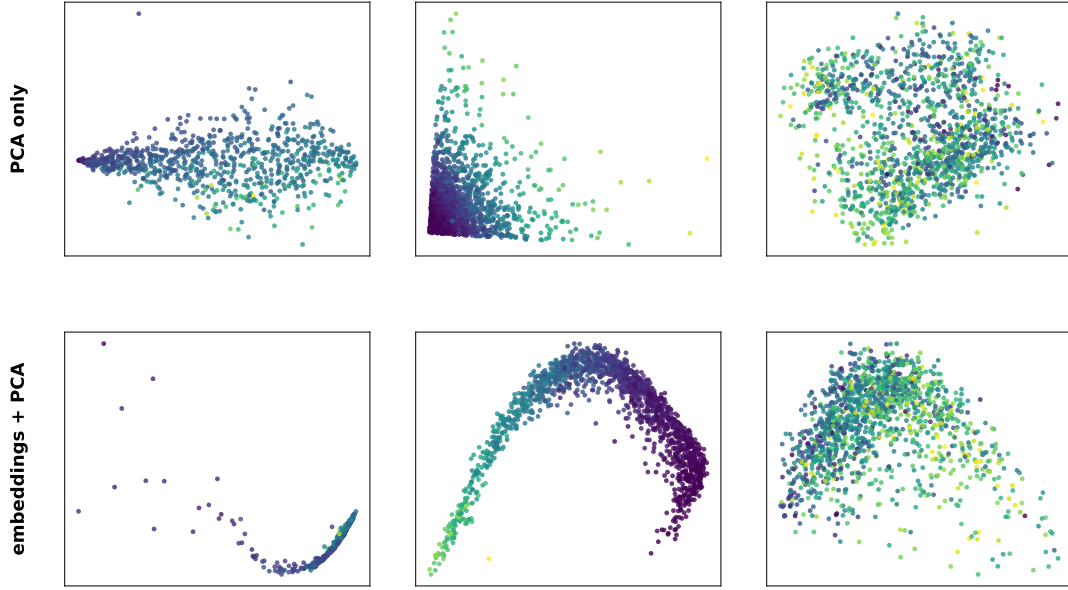


Figure 10. PCA visualization of the representations learned by Xiaomi-TabLDM on three classification datasets. The top row presents projections of the original feature space, while the bottom row presents projections of the corresponding row embeddings extracted from Xiaomi-TabLDM. Points denote individual samples and colors denote class labels. Compared with the raw inputs, the learned embeddings exhibit more organized structures that better reflect the class distribution.

evaluation regimes, while requiring substantially less computational cost than larger or heavily tuned alternatives such as TabFM and AutoGluon in relevant comparisons. These results indicate that improving tabular foundation models does not necessarily require scaling model size or computational cost alone; effective architectural design and capacity allocation can provide another practical path toward stronger and more efficient tabular learning systems.

The results demonstrate that Xiaomi-TabLDM provides a competitive balance of predictive performance, model capacity, and computational efficiency across heterogeneous tabular tasks. We hope these findings provide a useful basis for further exploration of scalable and efficient foundation-model architectures for structured data.

References

- [1] Alan Arazi, Eilam Shapira, and Roi Reichart. TabSTAR: A tabular foundation model for tabular data with text fields. In *Advances in Neural Information Processing Systems*, volume 38, 2025.
- [2] Sercan O. Arik and Tomas Pfister. TabNet: Attentive interpretable tabular learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(8):6679–6687, 2021.
- [3] Daniel Beaglehole, David Holzmüller, Adityanarayanan Radhakrishnan, and Mikhail Belkin. xRFM: Accurate, scalable, and interpretable feature learning models for tabular data. In *International Conference on Learning Representations*, 2026.
- [4] David Bonet, Marçal Comajoan Cara, Alvaro Calafell, Daniel Mas Montserrat, and Alexander G. Ioannidis. iLTM: Integrated large tabular model. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 186–197, 2026.
- [5] Mohamed Bouadi, Pratinav Seth, Aditya Tanna, and Vinay Kumar Sankarapu. Orion-MSP: Multi-scale sparse attention for tabular in-context learning, 2025.
- [6] Jintai Chen, Kuanlun Liao, Yanwen Fang, Danny Z. Chen, and Jian Wu. TabCaps: A capsule neural network for tabular data classification with BoW routing. In *International Conference on Learning Representations*, 2023.
- [7] Jintai Chen, Kuanlun Liao, Yao Wan, Danny Z. Chen, and Jian Wu. DANets: Deep abstract networks for tabular data classification and regression. *Proceedings of the AAAI Conference on Artificial Intelligence*, 36(4):3930–3938, 2022.
- [8] Jintai Chen, Jiahuan Yan, Qiyuan Chen, Danny Ziyi Chen, Jian Wu, and Jimeng Sun. ExcelFormer: A neural network surpassing GBDTs on tabular data, 2023.
- [9] Kuan-Yu Chen, Ping-Han Chiang, Hsin-Rung Chou, Ting-Wei Chen, and Tien-Hao Chang. Trompt: Towards a better deep neural network for tabular data. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 4392–4434. PMLR, 2023.
- [10] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM, 2016.
- [11] Damai Dai, Chengqi Deng, Chenggang Zhao, R.X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y.K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1280–1297. Association for Computational Linguistics, 2024.
- [12] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*, 2024.
- [13] Moonjung Eo, Min-Kook Suh, Hye-Seung Cho, Jiwon Kim, Seoyoon Kim, Sangjun Nam, and Soonyoung Lee. EXAONE Tabular 1.0: Technical report, 2026.
- [14] P. Erdős and A. Rényi. On random graphs i. *Publicationes Mathematicae Debrecen*, 6:290–297, 1959.
- [15] Nick Erickson, Jonas Mueller, Alexander Shirkov, Hang Zhang, Pedro Larroy, Mu Li, and Alexander Smola. AutoGluon-Tabular: Robust and accurate AutoML for structured data. In *ICML Workshop on Automated Machine Learning*, 2020.
- [16] Nick Erickson, Lennart Purucker, Andrej Tschalzev, David Holzmüller, Prateek Desai, David Salinas, and Frank Hutter. TabArena: A living benchmark for machine learning on tabular data. In *Advances in Neural Information Processing Systems*, volume 38, 2025.

- [17] Xi Fang, Weijie Xu, Fiona Anting Tan, Ziqing Hu, Jiani Zhang, Yanjun Qi, Srinivasan H. Sengamedu, and Christos Faloutsos. Large language models (LLMs) on tabular data: Prediction, generation, and understanding – a survey. *Transactions on Machine Learning Research*, 2024.
- [18] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- [19] Sebastian Felix Fischer, Matthias Feurer, and Bernd Bischl. OpenML-CTR23: A curated tabular regression benchmarking suite. In *AutoML Conference 2023 Workshop*, 2023.
- [20] Andreas Fuster, Paul Goldsmith-Pinkham, Tarun Ramadorai, and Ansgar Walther. Predictably unequal? the effects of machine learning on credit markets. *The Journal of Finance*, 77(1):5–47, 2022.
- [21] Anurag Garg, Muhammad Ali, Noah Hollmann, Lennart Purucker, Samuel Müller, and Frank Hutter. Real-TabPFN: Improving tabular foundation models via continued pre-training with real-world data, 2025.
- [22] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, 2006.
- [23] Yury Gorishniy, Akim Kotelnikov, and Artem Babenko. TabM: Advancing tabular deep learning with parameter-efficient ensembling. In *International Conference on Learning Representations*, 2025.
- [24] Yury Gorishniy, Ivan Rubachev, Nikolay Kartashev, Daniil Shlenskii, Akim Kotelnikov, and Artem Babenko. TabR: Tabular deep learning meets nearest neighbors. In *International Conference on Learning Representations*, 2024.
- [25] Léo Grinsztajn, Klemens Flöge, Oscar Key, Felix Birkel, Philipp Jund, et al. TabPFN-2.5: Advancing the state of the art in tabular foundation models, 2025.
- [26] Léo Grinsztajn, Klemens Flöge, Oscar Key, Felix Birkel, Philipp Jund, Brendan Roof, Mihir Manium, Shi Bin Hoo, Magnus Bühler, Anurag Garg, Dominik Safaric, Jake Robertson, Benjamin Jäger, Simone Alessi, Adrian Hayler, Vladyslav Moroshan, Lennart Purucker, Philipp Singer, Alan Arazi, Julien Siems, Jan Hendrik Metzen, Georg Grab, Nick Erickson, Siyuan Guo, Elliott Kalfon, Simon Bing, David Salinas, Clara Cornu, Lilly Charlotte Wehrhahn, Diana Kriuchkova, Kursat Kaya, Lydia Sidhoum, Marie Salmon, Jerry Chen, Madelon Hulsebos, Yann LeCun, Samuel Müller, Bernhard Schölkopf, Sauraj Gambhir, Noah Hollmann, and Frank Hutter. TabPFN-3: Technical report, 2026.
- [27] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs), 2016.
- [28] Noah Hollmann, Samuel Müller, Katharina Eggenberger, and Frank Hutter. TabPFN: A transformer that solves small tabular classification problems in a second. In *International Conference on Learning Representations*, 2023.
- [29] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025.
- [30] David Holzmüller, Léo Grinsztajn, and Ingo Steinwart. Better by default: Strong pre-tuned mlps and boosted trees on tabular data. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [31] Xin Huang, Ashish Khetan, Milan Cvitkovic, and Zohar Karnin. TabTransformer: Tabular data modeling using contextual embeddings, 2020.
- [32] Alan Jeffares, Tennison Liu, Jonathan Crabbé, Fergus Imrie, and Mihaela van der Schaar. TANGOS: Regularizing tabular neural networks through gradient orthogonalization and specialization. In *International Conference on Learning Representations*, 2023.
- [33] Alistair E. W. Johnson, Tom J. Pollard, Lu Shen, Li-wei H. Lehman, Mengling Feng, Mohammad Ghassemi, Benjamin Moody, Peter Szolovits, Leo Anthony Celi, and Roger G. Mark. MIMIC-III, a freely accessible critical care database. *Scientific Data*, 3:160035, 2016.

- [34] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. Technical blog, 2024.
- [35] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, pages 3146–3154, 2017.
- [36] Kimi Team, Guangyu Chen, Yu Zhang, Jianlin Su, Weixin Xu, Siyuan Pan, Yaoyu Wang, Yucheng Wang, Guanduo Chen, Bohong Yin, et al. Attention residuals, 2026.
- [37] Weihao Kong and Abhimanyu Das. Introducing TabFM: A zero-shot foundation model for tabular data. Google Research Blog, June 2026.
- [38] Peter M. Krafft, Meg Young, Michael Katell, Karen Huang, and Ghislain Bugingo. Defining AI in policy versus practice. In *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*, pages 72–78, 2020.
- [39] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3744–3753. PMLR, 2019.
- [40] Dmitry Lepikhin, HyounJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*, 2021.
- [41] Si-Yang Liu, Hao-Run Cai, Qi-Le Zhou, Huai-Hong Yin, Tao Zhou, Jun-Peng Jiang, and Han-Jia Ye. TALENT: A tabular analytics and learning toolbox. *Journal of Machine Learning Research*, 26(226):1–16, 2025.
- [42] Junwei Ma, Valentin Thomas, Rasa Hosseinzadeh, Alex Labach, Jesse C. Cresswell, Keyvan Golestan, Guangwei Yu, Anthony L. Caterini, and Maksims Volkovs. TabDPT: Scaling tabular foundation models on real data. In *Advances in Neural Information Processing Systems*, volume 38, 2025.
- [43] Duncan McElfresh, Sujay Khandagale, Jonathan Valverde, Vishak Prasad C, Benjamin Feuer, Chinmay Hegde, Ganesh Ramakrishnan, Micah Goldblum, and Colin White. When do neural nets outperform boosted trees on tabular data? In *Advances in Neural Information Processing Systems*, volume 36, pages 76336–76369, 2023.
- [44] Youssef Nader, Leon Sixt, and Tim Landgraf. DNNR: Differential nearest neighbors regression. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 16296–16317. PMLR, 2022.
- [45] Harsha Nori, Samuel Jenkins, Paul Koch, and Rich Caruana. InterpretML: A unified framework for machine learning interpretability, 2019.
- [46] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2 edition, 2009.
- [47] Jonas Peters, Dominik Janzing, and Bernhard Schölkopf. *Elements of Causal Inference: Foundations and Learning Algorithms*. MIT Press, 2017.
- [48] Sergei Popov, Stanislav Morozov, and Artem Babenko. Neural oblivious decision ensembles for deep learning on tabular data. In *International Conference on Learning Representations*, 2020.
- [49] Prior Labs. TabPFN-2.6. Model release, 2026. TabPFN model version 2.6.
- [50] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31, pages 6638–6648, 2018.

- [51] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. TabICL: A tabular foundation model for in-context learning on large data. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 50817–50847. PMLR, 2025.
- [52] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. TabICLv2: A better, faster, scalable, and open tabular foundation model, 2026.
- [53] Gowthami Somepalli, Avi Schwarzschild, Micah Goldblum, C. Bayan Bruss, and Tom Goldstein. SAINT: Improved neural networks for tabular data via row attention and contrastive pre-training. In *NeurIPS 2022 Workshops: TRL*, 2022.
- [54] Weiping Song, Chence Shi, Zhiping Xiao, Zhijian Duan, Yewen Xu, Ming Zhang, and Jian Tang. AutoInt: Automatic feature interaction learning via self-attentive neural networks. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, pages 1161–1170. ACM, 2019.
- [55] Boris Van Breugel and Mihaela Van Der Schaar. Position: Why tabular foundation models should be a research priority. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 48976–48993. PMLR, 2024.
- [56] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.
- [57] Ruoxi Wang, Rakesh Shivanna, Derek Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed H. Chi. DCN V2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. In *Proceedings of the Web Conference 2021*, pages 1785–1797. ACM, 2021.
- [58] Jing Wu, Suiyao Chen, Qi Zhao, Renat Sergazinov, Chen Li, Shengjie Liu, Chongchao Zhao, Tianpei Xie, Hanqing Guo, Cheng Ji, Daniel Cociorva, and Hakan Brunzell. SwitchTab: Switched autoencoders are effective tabular learners. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(14):15924–15933, 2024.
- [59] Jiahuan Yan, Jintai Chen, Yixuan Wu, Danny Z. Chen, and Jian Wu. T2G-FORMER: Organizing tabular features into relation graphs promotes heterogeneous feature interaction. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(9):10720–10728, 2023.
- [60] Han-Jia Ye, Huai-Hong Yin, De-Chuan Zhan, and Wei-Lun Chao. Revisiting nearest neighbor for tabular data: A deep tabular baseline two decades later. In *International Conference on Learning Representations*, 2025.
- [61] Wantao Yu, Chee Yew Wong, Roberto Chavez, and Mark A. Jacobs. Integrating big data analytics into supply chain finance: The roles of information processing and data-driven culture. *International Journal of Production Economics*, 236:108135, 2021.
- [62] Yuchen Zeng, Tuan Dinh, Wonjun Kang, and Andreas C. Mueller. TabFlex: Scaling tabular learning to millions with linear attention. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 74051–74079. PMLR, 2025.
- [63] Xingxuan Zhang, Gang Ren, Han Yu, Hao Yuan, Hui Wang, et al. LimiX: Unleashing structured-data modeling capability for generalist intelligence, 2025.
- [64] Xiyuan Zhang, Danielle Maddix Robinson, Junming Yin, Nick Erickson, Abdul Fatir Ansari, Boran Han, Shuai Zhang, Leman Akoglu, Christos Faloutsos, Michael W. Mahoney, Tony Hu, Huzefa Rangwala, George Karypis, and Yuyang Wang. Mitra: Mixed synthetic priors for enhancing tabular foundation models. In *Advances in Neural Information Processing Systems*, volume 38, pages 17831–17876, 2025.
- [65] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. ST-MoE: Designing stable and transferable sparse expert models, 2022.

Appendix

A Contributions and Acknowledgments

We express our sincere gratitude to all contributors for their dedication to the design, implementation, experimentation, and documentation of Xiaomi-TabLDM. Their collaborative efforts across data construction, model architecture, training, system integration, and empirical evaluation were instrumental to the success of this work and the release of the accompanying model and codebase. The contributors to this work are listed as follows:

Core Contributors: Penghui Wang[†], Wei Liu[†], Hong Wang, Chengyue Huang, Yuxi Sun, Zirui Wang, Hongming Huang, Quan Wang, Chunxiao Liu*, Erli Meng, Bin Wang

Contributors: Zhenwei Xin, Ping Hou, Jie Yu

[†] Equal contribution

* Corresponding author

A.1 Architectural Hyperparameters

The released Xiaomi-TabLDM classifier and regressor checkpoints share all architectural hyperparameters and the sparse MoE configuration, differing only in the task-specific target encoder and the final linear layer of the output decoder. These hyperparameters are summarized in the tables below (Tables 4–8).

Table 4. Column-wise feature embedding.

Hyperparameter	Value	Description
embed_dim	128	Base embedding dimension used model-wide
col_feature_group_size	3	Features per circular-shift group
global_dilation	adaptive	Offset scheme for the second column stream
global_max_span	32	Target span $s_{\max}$ for the second stream
col_num_blocks	3	Induced self-attention blocks
col_nhead	8	Attention heads per block
col_num_inds	128	Inducing points per column

Table 5. Row-wise feature aggregation.

Hyperparameter	Value	Description
row_num_blocks	3	Transformer blocks
row_nhead	8	Attention heads per block
row_num_cls	4	CLS tokens aggregated per row
use_rope	True	Rotary positional embeddings (RoPE) enabled
row_rope_base	100 000	RoPE base frequency θ
block_size	4	AttnRes residual block size
attnres_stride	4	Depth stride at which AttnRes is applied

A.2 MoE auxiliary losses

We regularize routing with two auxiliary terms per MoE layer: a Switch-style load-balance loss[18] that spreads tokens evenly over the M_R routed experts, and a router z -loss[65] that keeps the logits bounded. Let $z_{t,i}$ be the

Table 6. Dataset-wise in-context learning.

Hyperparameter	Value	Description
icl_emsized (derived)	512	$\text{embed_dim} \times \text{row_num_cls} = 128 \times 4$
icl_num_blocks	24	Transformer blocks
icl_nhead	8	Attention heads per block
ff_factor	2	Feed-forward expansion factor
block_size	4	AttnRes residual block size
attnres_stride	4	Depth stride at which AttnRes is applied

Table 7. The output decoder uses a two-layer MLP with a GELU activation [27]. $\text{Linear}(512, 1024) \rightarrow \text{GELU} \rightarrow \text{Linear}(1024, \text{out_dim})$. Only `out_dim` and the target encoder differ between the two models.

Hyperparameter	Classifier	Regressor
out_dim	10	999
target encoder	OneHotAndLinear(10, 512)	Linear(1, 512)

Table 8. Sparse MoE. Replaces the dense FFN in selected ICL blocks. Each expert is a 2-layer MLP with the same hidden width as the dense FFN it replaces, $\text{icl_emsized} \times \text{ff_factor} = 512 \times 2 = 1024$.

Hyperparameter	Value	Description
icl_moe_layers	last_8	MoE blocks; final 8 of 24
icl_moe_num_experts	2	Routed experts per layer
icl_moe_top_k	1	Experts activated per token
icl_moe_num_shared_experts	1	Always-on shared expert
expert_hidden_dim (derived)	1024	Per-expert hidden width, $\text{icl_emsized} \times \text{ff_factor}$
icl_moe_init_from_dense	True	Experts initialized from the frozen dense FFN
icl_moe_router_jitter	0.0	Router input jitter (disabled)

router logit of expert i on row token t and T is the total number of row tokens,

$$\mathcal{L}_{\text{bal}} = M_R \sum_{i=1}^{M_R} f_i P_i, \quad \mathcal{L}_z = \frac{1}{T} \sum_{t=1}^T \left(\log \sum_{i=1}^{M_R} e^{z_{t,i}} \right)^2 \quad (3)$$

where f_i is the fraction of tokens routed to expert i and P_i its mean gate probability, so $\mathcal{L}_{\text{bal}}$ is minimized under uniform load. Summed over MoE layers and scaled by the coefficients in Table 9, they are added to the task loss $\mathcal{L}_{\text{task}}$ (cross-entropy for the classifier, pinball loss for the regressor),

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda(\alpha_{\text{bal}} \mathcal{L}_{\text{bal}} + \alpha_z \mathcal{L}_z) \quad (4)$$

Table 9. MoE auxiliary losses. Both terms are computed per MoE layer, summed over layers, scaled by `icl_moe_aux_loss_weight`, and added to the task loss.

Hyperparameter	Symbol	Value	Description
<code>icl_moe_router_z_loss_coef</code>	α_z	1e-3	Router z-loss weight
<code>icl_moe_load_balance_loss_coef</code>	α_{bal}	1e-2	Load-balance weight
<code>icl_moe_aux_loss_weight</code>	λ	1.0	Auxiliary loss multiplier

A.3 Model parameter counts

As shown in Table 10, we list the parameter counts of the Xiaomi-TabLDM classification and regression models. Total parameters count all weights, while active parameters count those actually used in a single forward pass, determined by the sparse MoE configuration.

Table 10. Parameter counts of the released Xiaomi-TabLDM checkpoints.

Model	Type	Total params	Active params
classifier	Classification (<code>max_classes</code> =10)	70.08 M	61.67 M
regressor	Regression (<code>quantiles</code> =999)	71.08 M	62.68 M

A.4 Many-class classification

Xiaomi-TabLDM is pretrained with at most `max_classes` = 10 classes. For tasks with $C > 10$ classes, the mixed-radix ensembling of TabICLv2[52] is applied: the labels are re-encoded into D mixed-radix digits over balanced bases, and the column-wise transformer is run once per digit view with the outputs averaged, thereby supporting an arbitrary number of classes without retraining. Formally, balanced bases $[k_0, \dots, k_{D-1}]$ are chosen with every $k_i \leq 10$ and $\prod_{i=0}^{D-1} k_i \geq C$, and each label is rewritten into D mixed-radix digits,

$$y^{(i)} = \left\lfloor y / \prod_{j>i} k_j \right\rfloor \bmod k_i, \quad i = 0, \dots, D-1. \quad (5)$$

Mixed-radix ensembling acts at the column-wise feature embedding stage, while hierarchical classification at the dataset-wise ICL stage handles the remaining many-class structure.

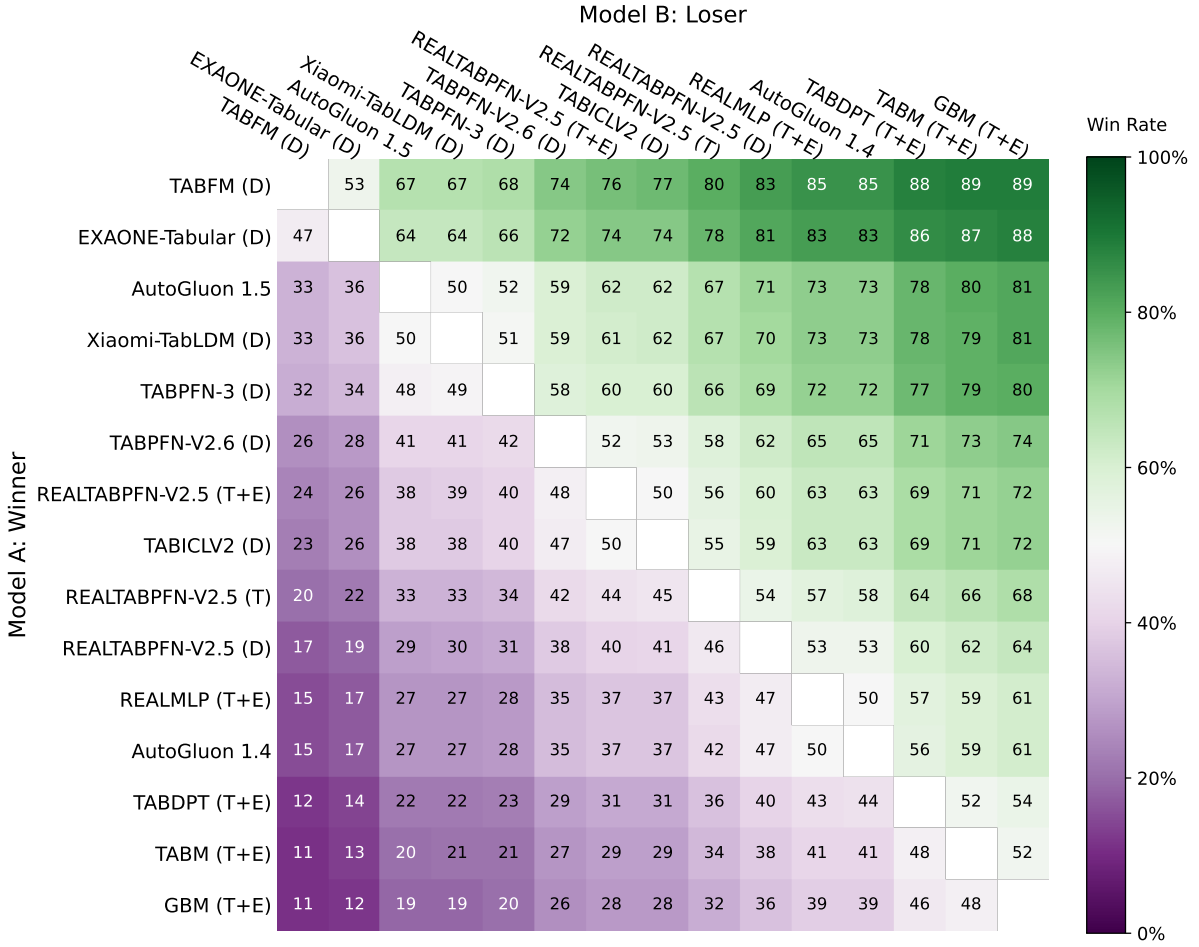


Figure 11. Pairwise win-rate matrix on the full TabArena benchmark. Each entry (i, j) reports the proportion of instances on which model i outperforms model j . Diagonal entries denote self-comparison (50% by definition).

B Results

Our comparisons cover a broad range of tabular learning approaches. For tabular foundation models, we include TabFM [37], EXAONE-Tabular [13], TabPFN-3 [26], TabPFN-2.6 [49], RealTabPFN [21], TabICLv2 [52], TabDPT [42], LimiX [63], iLTM [4], TabSTAR [1], TabFlex [62], and OrionMSP [5]. We further compare against strong deep and feature-learning methods, including RealMLP [30], ModernNCA [60], TabM [23], and xRFM [3], as well as tree-based and classical baselines such as XGBoost [10], CatBoost [50], LightGBM [35], and EBM [45]. AutoGluon [15] is included as a strong AutoML baseline. The complete set of evaluated methods and configurations, including additional classical and neural baselines, follows the TabArena benchmark [16] and is reported in the appendix.

B.1 Details of TabArena

B.1.1 Evaluation Metrics

We reuse the official TabArena [16] evaluation metrics and evaluation code for generating the TabArena plots and leaderboard tables.

Elo. Following TabArena, we evaluate models using the Elo rating system. Elo is based on pairwise model comparisons, where the difference between two models’ ratings determines their expected win probability. A 400-point Elo difference corresponds to an expected win probability of approximately 91% for the higher-rated model. Following the official TabArena protocol, we calibrate an Elo score of 1000 to the default RandomForest configuration and perform 200 bootstrap rounds to estimate 95% confidence intervals. For task-level comparisons, TabArena uses ROC-AUC for binary classification, log-loss for multiclass classification, and RMSE for regression.

Improvability. We additionally report the Improvability metric introduced in TabArena. For a dataset i , it measures the relative reduction in error required for a method to match the best-performing method on that dataset:

$$\text{Improvability}_i = \frac{\text{err}_i - \text{best_err}_i}{\text{err}_i} \times 100\%. \quad (6)$$

The final Improvability score is averaged across datasets. Lower values are better, with 0% indicating performance equal to the best observed method on every dataset.

Dataset wins. We also report the number of dataset wins, which measures how often a method achieves the best performance among the compared methods. Together with Elo and Improvability, this provides a complementary view of both average performance and task-level dominance.

B.1.2 Tuning and Efficiency Plots

We follow the official TabArena plotting protocol to analyze the effect of tuning and ensembling. For methods with multiple configurations, the plots compare the default configuration with tuned and tuned-and-ensembled variants. The tuning trajectories are constructed from increasingly large ensembles of sampled configurations, following the TabArena evaluation procedure.

In addition to predictive performance, we report computational efficiency using the median training and inference time per 1K samples. These results allow us to compare not only the achievable performance of different methods, but also the additional computational cost introduced by dataset-specific tuning and ensembling. In particular, they highlight the trade-off between strong default performance and substantially more expensive tuned pipelines.

B.1.3 TabArena Leaderboard Tables

Tables 11 and 12 report the detailed TabArena leaderboard results on the full benchmark and the regression subset, respectively.

Across all 51 datasets and 816 tasks, Xiaomi-TabLDM achieves an Elo score of **1659**, ranking **fourth** by Elo point estimate. Its performance is nearly tied with AutoGluon 1.5 extreme (1662) and surpasses strong recent tabular foundation models including TabPFN-3 (1650), TabPFN-2.6 (1596), RealTabPFN-2.5 with tuning and ensembling (1579), and TabICLv2 (1576). Xiaomi-TabLDM also achieves **3.1 dataset wins**, exceeding TabPFN-3 and the other recent tabular foundation models except TabFM and EXAONE-Tabular. These results demonstrate that Xiaomi-TabLDM remains highly competitive across the full TabArena benchmark.

The strongest relative performance of Xiaomi-TabLDM is observed on regression. Across the 13 regression datasets, Xiaomi-TabLDM reaches an Elo score of **1900**, ranking **second** behind only TabFM (2019). It outperforms EXAONE-Tabular (1885), TabPFN-3 (1800), AutoGluon 1.5 extreme (1776), TabPFN-2.6 (1741), RealTabPFN-2.5 with tuning and ensembling (1736), and TabICLv2 (1679). In addition, Xiaomi-TabLDM achieves **1.8 dataset wins** and an improvability of only **1.7%**, the second-best result behind TabFM (1.6%).

Notably, this regression performance is achieved with substantially lower computational cost than TabFM. Xiaomi-TabLDM requires 6.99 s of training time and 3.12 s of prediction time per 1K samples, compared with 38.85 s and 9.67 s for TabFM, corresponding to approximately **82% less training time** and **68% less prediction time**. Overall, the TabArena leaderboards show that Xiaomi-TabLDM combines competitive performance across the full benchmark with particularly strong regression performance and a favorable performance–efficiency trade-off.

B.2 Details on TALENT benchmark

We evaluate Xiaomi-TabLDM on TALENT (Tabular Analytics and LEarNing Toolbox) [41], a comprehensive benchmark and evaluation framework for tabular prediction. TALENT brings together a broad range of classical machine-learning and deep-learning methods under a unified evaluation pipeline, with standardized preprocessing and model interfaces to facilitate consistent comparison across heterogeneous tabular datasets. The benchmark covers both classification and regression tasks with substantial variation in dataset size, feature dimensionality, class structure, and feature composition.

For classification, TALENT supports multiple evaluation metrics, including Accuracy, F1-score, Log Loss, and AUC. In our experiments shown in Figure 6, we use **Accuracy** as the primary classification metric and compute the average rank of each model across datasets based on its per-dataset Accuracy. Thus, the reported classification rank reflects how consistently a model performs relative to other methods across the benchmark, rather than its average Accuracy value alone.

B.3 Prior visualizations

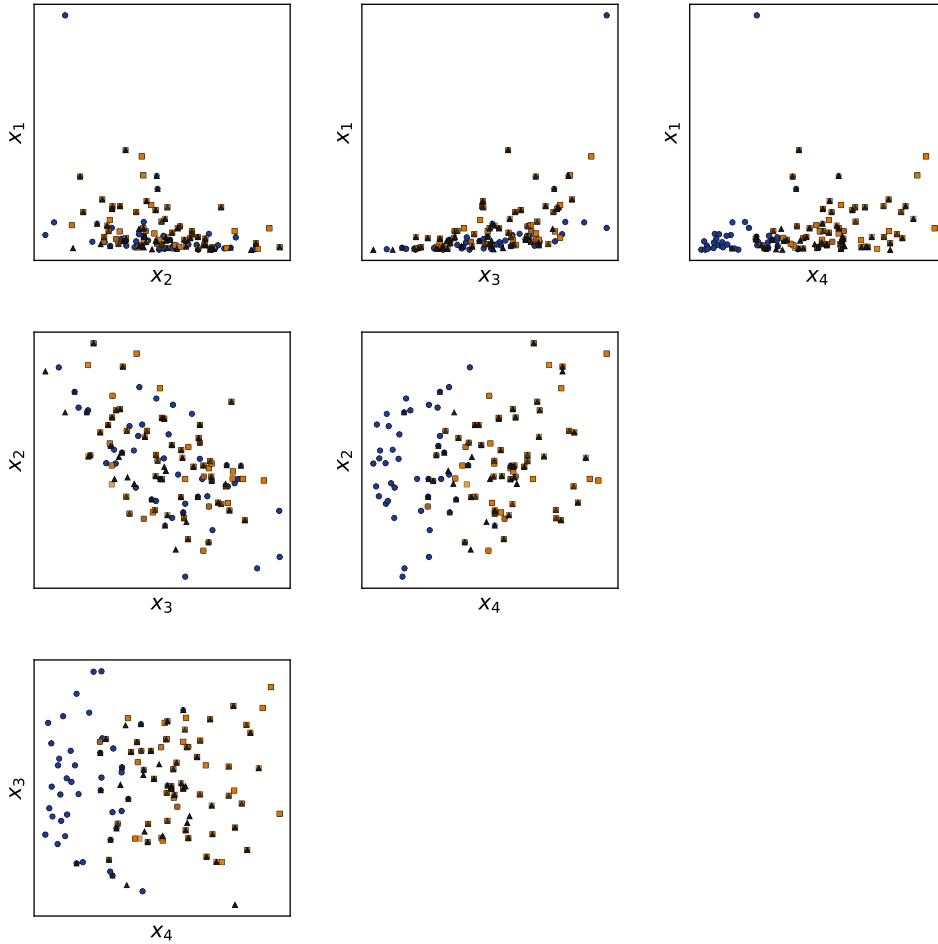


Figure 12. Example classification dataset sampled from the prior with four covariates. Each subplot in row i , column j visualizes covariates i and $j + 1$, with colors denoting the target classes.

To illustrate the functional diversity introduced by our new combiner mechanisms, we visualize representative relationships sampled from the SCM prior in Fig. 13. The individual edge functions in Fig. 13a span a broad range of behaviors, including smooth nonlinear mappings, localized responses, piecewise transitions, and highly

irregular surfaces. These functions serve as basic building blocks for constructing dependencies between variables in the sampled SCMs.

More importantly, composing and mixing multiple edge functions further expands the space of functional relationships represented by the prior. As shown in Fig. 13b, the resulting functions can exhibit substantially richer structures, combining sharp transitions, local nonlinearities, and heterogeneous behaviors within a single relationship. This compositional design allows the synthetic prior to cover a wider variety of dependencies than relying on a small set of predefined functional forms alone. Although the actual mechanisms operate on inputs of varying dimensionality, we restrict the visualization to two-dimensional input spaces for clarity.

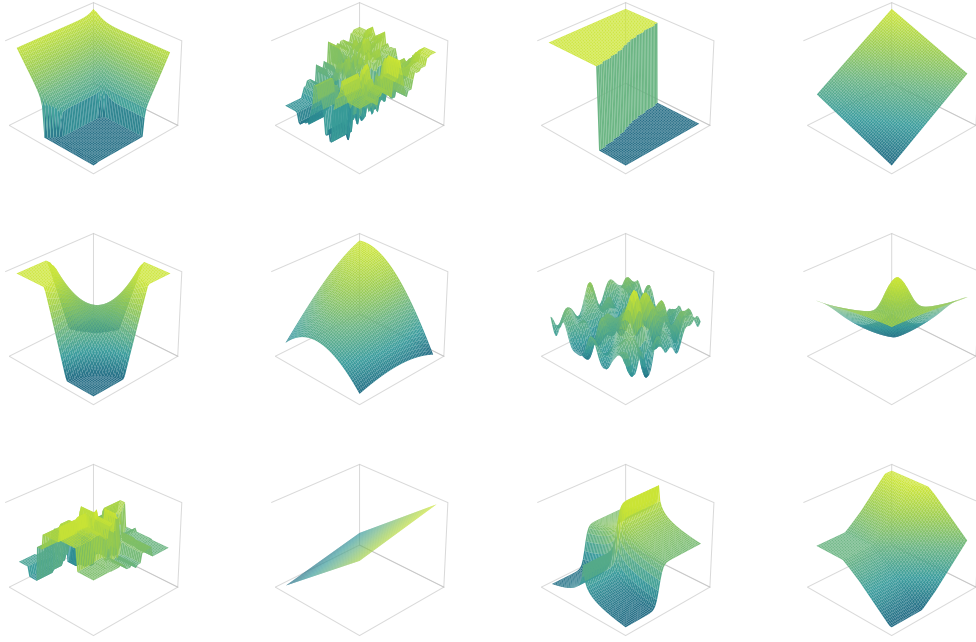
Table 11. Overall performance on TabArena. We compare models in terms of Elo, number of wins, improvability, and training and prediction time. Here **D** denotes the default (untuned) model, **T** the fine-tuned model, and **T+E** ensembling after fine-tuning.

Model	Elo ($\uparrow$)	#wins ($\uparrow$)	Improvability ($\downarrow$)	Train time per 1K [s]	Predict time per 1K [s]
TABFM (D)	1782 _{-102,+109}	21.1	4.4%	38.85	10.52
EXAONE-Tabular (D)	1762 _{-61,+85}	5.4	7.9%	9.00	2.10
AutoGluon 1.5 (extreme, 4h)	1662 _{-62,+75}	3.6	8.3%	289.07	4.03
Xiaomi-TabLDM (D)	1659 _{-55,+86}	3.1	9.7%	10.92	4.18
TabPFN-3 (D)	1650 _{-55,+76}	1.8	9.6%	4.97	0.58
TabPFN-2.6 (D)	1596 _{-47,+68}	0.3	11.0%	5.48	0.55
RealTabPFN-2.5 (T+E)	1579 _{-58,+70}	0.6	10.7%	2040.22	8.91
TabICLv2 (D)	1576 _{-58,+67}	1.3	10.4%	4.02	0.38
RealTabPFN-2.5 (T)	1538 _{-51,+62}	0.5	11.4%	2040.22	1.22
RealTabPFN-2.5 (D)	1510 _{-44,+57}	0.1	11.9%	5.81	0.64
RealMLP (T+E)	1486 _{-44,+55}	0.2	13.2%	2950.72	11.97
AutoGluon 1.4 (best, 4h)	1486 _{-47,+54}	0.0	13.1%	1735.72	2.56
TabDPT (T+E)	1441 _{-48,+60}	1.7	14.1%	4910.38	286.54
TabM (T+E)	1425 _{-40,+49}	0.2	14.5%	3286.61	1.47
LightGBM (T+E)	1411 _{-31,+32}	0.0	15.3%	417.05	2.64
RealMLP (T)	1410 _{-46,+47}	0.1	14.5%	2950.72	0.66
CatBoost (T+E)	1396 _{-35,+40}	0.1	14.9%	1658.43	0.65
TabDPT (T)	1388 _{-54,+54}	0.2	15.2%	4910.38	39.96
iLTM (T+E)	1388 _{-43,+45}	0.1	15.6%	12685.08	464.37
CatBoost (T)	1386 _{-39,+39}	0.4	15.1%	1658.43	0.08
TabM (T)	1373 _{-41,+48}	0.1	15.3%	3286.61	0.17
ModernNCA (T+E)	1370 _{-53,+70}	0.7	15.9%	4621.67	8.14
LightGBM (T)	1368 _{-28,+29}	0.0	15.9%	417.05	0.33
XGBoost (T+E)	1358 _{-34,+32}	0.0	16.0%	693.49	1.69
CatBoost (D)	1351 _{-40,+36}	0.0	15.8%	6.83	0.08
LimiX (D)	1347 _{-60,+72}	1.5	15.8%	26.46	6.24
ModernNCA (T)	1341 _{-39,+39}	0.3	16.3%	4621.67	0.47
XGBoost (T)	1336 _{-33,+31}	0.0	16.3%	693.49	0.31
xRFM (T+E)	1335 _{-43,+46}	0.0	16.6%	846.89	2.55
TabPFNV2 (T+E)	1328 _{-65,+66}	0.2	16.9%	3031.50	21.44
Mitra (D)	1316 _{-65,+62}	0.3	17.4%	87.65	2.50
TabDPT (D)	1314 _{-54,+65}	0.1	17.5%	47.65	43.74
TabICL (D)	1308 _{-58,+50}	0.0	17.3%	6.63	1.48
xRFM (T)	1291 _{-39,+44}	0.1	17.7%	846.89	0.13
TabM (D)	1286 _{-42,+47}	0.1	17.3%	10.50	0.13
iLTM (T)	1285 _{-33,+38}	0.1	17.4%	12685.08	62.13
TorchMLP (T+E)	1275 _{-45,+49}	0.0	17.4%	2875.52	1.95

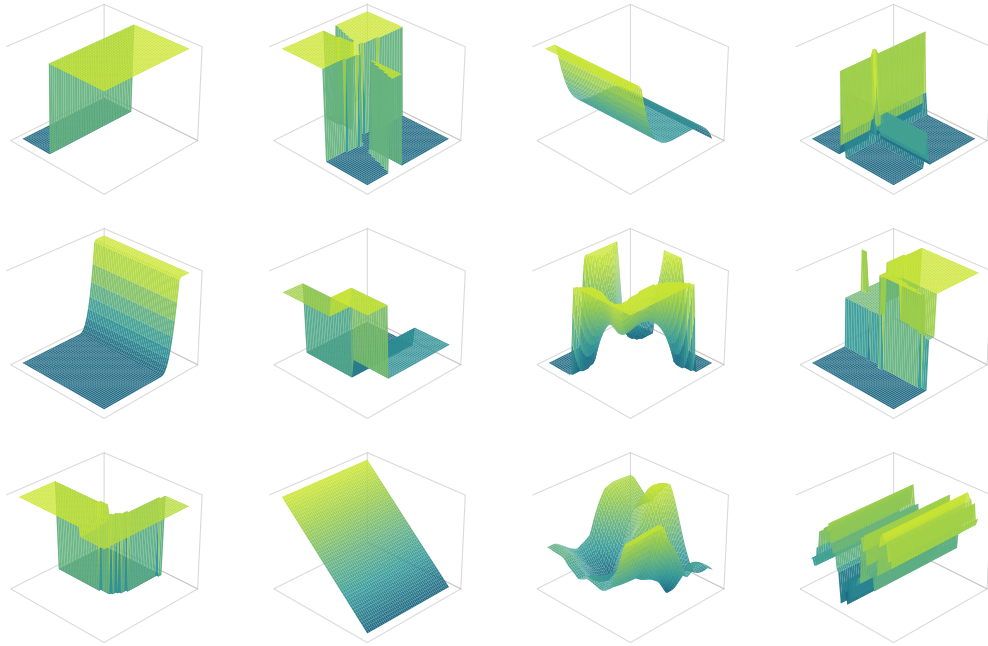
Continued on next page

Table 11 continued from previous page

Model	Elo ($\uparrow$)	#wins ($\uparrow$)	Improva- bility ($\downarrow$)	Train time per 1K [s]	Predict time per 1K [s]
SAP-RPT-OSS (D)	1272 _{-54,+54}	0.8	18.5%	14.11	2.07
TabPFNV2 (T)	1272 _{-57,+58}	0.2	18.4%	3031.50	0.46
BetaTabPFN (D)	1271 _{-50,+55}	0.1	18.8%	205.88	1.34
EBM (T+E)	1256 _{-39,+38}	0.0	18.9%	2931.75	0.42
TabPFNV2 (D)	1247 _{-62,+64}	0.3	19.1%	3.36	0.31
ModernNCA (D)	1240 _{-38,+40}	0.3	19.4%	14.87	0.31
EBM (T)	1222 _{-42,+44}	0.0	19.6%	2931.75	0.05
RealMLP (D)	1222 _{-38,+35}	0.1	18.8%	10.06	1.69
XGBoost (D)	1209 _{-38,+36}	0.0	19.0%	1.94	0.12
TorchMLP (T)	1204 _{-44,+42}	0.0	19.1%	2875.52	0.13
ExtraTrees (T+E)	1198 _{-43,+42}	0.1	20.2%	183.02	0.76
FastaiMLP (T+E)	1196 _{-54,+49}	0.2	19.9%	593.24	4.47
EBM (D)	1192 _{-51,+46}	0.1	20.5%	7.33	0.05
LightGBM (D)	1183 _{-32,+31}	0.0	19.6%	1.96	0.14
RandomForest (T+E)	1173 _{-43,+51}	0.1	21.1%	373.24	0.77
ExtraTrees (T)	1165 _{-47,+42}	0.1	21.1%	183.02	0.09
FastaiMLP (T)	1138 _{-53,+50}	0.1	21.3%	593.24	0.31
RandomForest (T)	1136 _{-41,+50}	0.2	21.8%	373.24	0.09
iLTM (D)	1089 _{-52,+46}	0.2	23.3%	296.64	68.17
TabSTAR (T)	1088 _{-80,+74}	0.9	25.8%	28729.74	4.27
TabSTAR (T+E)	1087 _{-80,+78}	1.1	25.8%	28729.74	18.81
OrionMSP (D)	1087 _{-49,+52}	0.0	23.9%	13.12	2.52
PerpetualBooster (T+E)	1087 _{-46,+45}	0.0	26.6%	185.31	0.60
TorchMLP (D)	1073 _{-48,+35}	0.1	23.0%	9.99	0.13
PerpetualBooster (T)	1050 _{-44,+45}	0.0	28.0%	185.31	0.26
xRFM (D)	1038 _{-68,+58}	0.0	26.7%	3.23	0.92
TabFlex (D)	1010 _{-69,+60}	0.1	27.8%	0.79	0.12
FastaiMLP (D)	1003 _{-60,+55}	0.1	25.9%	2.86	0.37
RandomForest (D)	1000 _{-43,+41}	0.0	26.5%	0.43	0.05
KNN (T+E)	990 _{-83,+59}	0.2	28.0%	129.08	1.80
TabSTAR (D)	988 _{-96,+87}	0.4	30.7%	384.75	5.35
ExtraTrees (D)	980 _{-67,+55}	0.0	27.8%	0.25	0.05
Linear (T+E)	956 _{-101,+60}	0.1	33.9%	237.63	0.42
Linear (T)	931 _{-108,+67}	0.0	34.4%	237.63	0.08
PerpetualBooster (D)	930 _{-61,+42}	0.1	31.9%	27.32	0.03
KNN (T)	885 _{-95,+61}	0.1	33.0%	129.08	0.18
Linear (D)	856 _{-113,+68}	0.1	37.0%	1.19	0.12
KNN (D)	644 _{-90,+79}	0.1	46.1%	0.19	0.04



(a) Examples of individual edge functions generated by the new combiner mechanisms.



(b) Examples of more complex functional relationships obtained by composing and mixing multiple edge functions.

Figure 13. Visualization of functional relationships generated by the new combiner mechanisms in our SCM prior. The top panel shows individual edge functions, while the bottom panel illustrates more complex relationships constructed by composing and mixing these functions. Although mechanisms in the prior may have varying input dimensionality, we visualize them on a two-dimensional grid for clarity.

Table 12. Performance on the regression subset of TabArena, covering 13 datasets. We compare models in terms of Elo, number of wins, improvability, and training and prediction time. Xiaomi-TabLDM achieves the second-highest Elo while maintaining substantially lower computational cost than most tuned and ensembled baselines. Here **D** denotes the default (untuned) model, **T** the fine-tuned model, and **T+E** ensembling after fine-tuning.

Model	Elo ($\uparrow$)	#wins ($\uparrow$)	Improvability ($\downarrow$)	Train time per 1K [s]	Predict time per 1K [s]
TABFM (D)	2019 _{-127,+181}	3.7	1.6%	38.85	9.67
Xiaomi-TabLDM (D)	1900 _{-148,+278}	1.8	1.7%	6.99	3.12
EXAONE-Tabular (D)	1885 _{-110,+155}	1.5	3.0%	13.71	2.58
TabPFN-3 (D)	1800 _{-131,+211}	0.9	2.4%	3.87	0.42
AutoGluon 1.5 (extreme, 4h)	1776 _{-96,+137}	1.3	3.9%	335.03	4.33
TabPFN-2.6 (D)	1741 _{-56,+104}	0.1	4.0%	8.52	0.70
RealTabPFN-2.5 (T+E)	1736 _{-103,+153}	0.2	3.4%	1709.05	8.12
TabDPT (T+E)	1722 _{-90,+160}	1.5	4.3%	4786.60	239.30
TabICLv2 (D)	1679 _{-142,+242}	0.5	4.0%	2.10	0.25
TabDPT (T)	1670 _{-73,+128}	0.0	4.7%	4786.60	38.50
RealMLP (T+E)	1650 _{-67,+111}	0.1	5.1%	3995.01	10.05
RealTabPFN-2.5 (T)	1624 _{-117,+154}	0.2	4.1%	1709.05	0.81
AutoGluon 1.4 (best, 4h)	1592 _{-90,+117}	0.0	6.4%	1866.35	6.07
TabDPT (D)	1584 _{-64,+137}	0.0	5.6%	46.62	39.21
RealMLP (T)	1547 _{-82,+105}	0.0	6.0%	3995.01	0.84
RealTabPFN-2.5 (D)	1534 _{-106,+141}	0.0	5.6%	7.04	0.51
ModernNCA (T+E)	1531 _{-118,+145}	0.6	7.3%	3779.70	7.69
CatBoost (T+E)	1482 _{-65,+103}	0.0	8.0%	3555.27	0.96
LightGBM (T+E)	1474 _{-83,+89}	0.0	8.4%	700.19	9.32
xRFM (T+E)	1464 _{-95,+112}	0.0	7.5%	714.50	1.38
CatBoost (T)	1462 _{-66,+102}	0.0	8.1%	3555.27	0.10
TabM (T+E)	1442 _{-85,+130}	0.0	6.9%	4160.58	1.41
iLTM (T+E)	1438 _{-46,+68}	0.0	9.3%	12685.08	321.74
LightGBM (T)	1413 _{-75,+91}	0.0	9.0%	700.19	0.97
XGBoost (T+E)	1402 _{-47,+64}	0.0	9.0%	834.93	2.61
xRFM (T)	1384 _{-81,+89}	0.0	8.2%	714.50	0.10
XGBoost (T)	1382 _{-53,+67}	0.0	9.1%	834.93	0.39
CatBoost (D)	1369 _{-89,+93}	0.0	9.7%	10.89	0.09
ModernNCA (T)	1369 _{-87,+105}	0.0	9.3%	3779.70	0.40
TabM (T)	1366 _{-97,+122}	0.0	7.9%	4160.58	0.17
iLTM (T)	1333 _{-57,+75}	0.0	9.3%	12685.08	59.10
TabPFNV2 (T+E)	1318 _{-116,+163}	0.0	8.7%	4223.87	27.54
TabM (D)	1277 _{-109,+113}	0.0	9.5%	13.32	0.13
ModernNCA (D)	1274 _{-68,+81}	0.0	10.8%	15.50	0.30
Mitra (D)	1264 _{-101,+126}	0.1	10.3%	71.06	1.85
SAP-RPT-OSS (D)	1258 _{-140,+144}	0.1	10.8%	20.24	6.62
LimiX (D)	1246 _{-154,+167}	0.1	10.4%	74.68	19.76
TorchMLP (T+E)	1236 _{-100,+110}	0.0	11.0%	4608.59	1.23
TabPFNV2 (T)	1232 _{-127,+153}	0.0	9.7%	4223.87	0.45
RealMLP (D)	1226 _{-91,+107}	0.0	10.5%	8.90	1.64
ExtraTrees (T+E)	1210 _{-94,+100}	0.0	13.2%	158.25	0.84
LightGBM (D)	1206 _{-41,+41}	0.0	11.4%	2.11	0.27
XGBoost (D)	1189 _{-77,+83}	0.0	12.0%	2.24	0.24
TabPFNV2 (D)	1184 _{-143,+133}	0.0	11.1%	2.80	0.31

Continued on next page

Table 12 continued from previous page

Model	Elo ($\uparrow$)	#wins ($\uparrow$)	Improva- bility ($\downarrow$)	Train time per 1K [s]	Predict time per 1K [s]
ExtraTrees (T)	1184 _{-88,+92}	0.0	13.4%	158.25	0.15
TorchMLP (T)	1180 _{-96,+95}	0.0	11.7%	4608.59	0.10
PerpetualBooster (T+E)	1180 _{-85,+73}	0.0	13.2%	162.38	0.36
RandomForest (T+E)	1162 _{-61,+62}	0.0	14.0%	515.75	0.77
xRFM (D)	1147 _{-109,+110}	0.0	13.8%	2.45	0.74
EBM (T+E)	1144 _{-166,+124}	0.0	14.5%	2931.75	0.29
PerpetualBooster (T)	1128 _{-88,+57}	0.0	14.3%	162.38	0.17
RandomForest (T)	1120 _{-76,+63}	0.0	14.5%	515.75	0.12
EBM (T)	1101 _{-173,+126}	0.0	15.0%	2931.75	0.03
ExtraTrees (D)	1069 _{-107,+94}	0.0	15.3%	0.47	0.06
EBM (D)	1042 _{-173,+119}	0.0	15.9%	8.47	0.04
TorchMLP (D)	1027 _{-115,+81}	0.0	15.1%	20.49	0.08
FastaiMLP (T+E)	1026 _{-113,+103}	0.0	15.3%	540.14	2.67
FastaiMLP (T)	979 _{-111,+98}	0.0	15.8%	540.14	0.32
TabSTAR (T+E)	951 _{-284,+224}	0.1	23.5%	28729.74	19.90
PerpetualBooster (D)	943 _{-117,+79}	0.0	17.7%	27.32	0.02
TabSTAR (T)	937 _{-294,+225}	0.1	23.7%	28729.74	5.24
KNN (T+E)	882 _{-154,+140}	0.0	21.0%	92.55	0.90
iLTM (D)	871 _{-108,+67}	0.0	19.0%	357.02	81.61
FastaiMLP (D)	864 _{-160,+111}	0.0	20.0%	2.60	0.39
TabSTAR (D)	832 _{-345,+237}	0.1	26.6%	323.52	4.93
KNN (T)	782 _{-174,+145}	0.0	23.3%	92.55	0.05
KNN (D)	680 _{-246,+170}	0.0	30.4%	0.19	0.04
Linear (T+E)	502 _{-376,+127}	0.0	37.4%	193.98	0.17
Linear (T)	469 _{-447,+147}	0.0	37.6%	193.98	0.07
Linear (D)	295 _{-413,+146}	0.0	40.0%	0.95	0.10

C Sensitivity Analysis

Although a number of mature benchmarks now enable systematic comparisons across different tabular foundation models, existing evaluation frameworks remain insufficient for fully characterizing how these models respond to external perturbations. To fill this gap, this section focuses on evaluating the robustness of tabular regression models to exogenous noise. In the overall factorial experimental design, we treat graph density and node-level signal-to-noise ratio (SNR) as two explicit and balanced experimental factors. For each matched base configuration, we fix the graph structure, structural functions, SNR vector, and feature permutation, and vary only the noise condition. Performance differences across noise conditions can therefore be attributed more directly to a model’s sensitivity to exogenous noise, rather than to other uncontrolled variations in the data-generating process. The experimental results show that our proposed model maintains a substantial relative advantage across a wide range of noise distributions, SNR ranges, and graph density conditions, demonstrating strong robustness to exogenous perturbations.

C.1 Experimental Design and Data Generation

Each dataset is generated by a structural causal model with 32 observed variables. Of these, 31 variables serve as predictive features, and the last node in the sampled topological order serves as the regression target. No latent variables are introduced, so the evaluation is not additionally affected by unobserved variables or latent confounders. Although all variables are observable, the underlying graph structure, structural functions, and noise mechanisms remain unknown to the evaluated models.

Graphs are sampled using an ordered Erdős–Rényi construction [14]. Given a topological order, each candidate forward edge consistent with that order is added to the graph independently with probability p , so acyclicity is guaranteed by construction. Graph density is the first experimental factor and comprises two levels. In the sparse setting, $p \sim \mathcal{U}[0.04, 0.10]$, corresponding to an expected average in-degree of approximately 0.62–1.55 and thus yielding relatively many root nodes. In the dense setting, $p \sim \mathcal{U}[0.25, 0.40]$, corresponding to an expected average in-degree of approximately 3.88–6.20, with a substantially smaller number of root nodes. A clear gap is kept between the two intervals, so the sparse and dense settings form two clearly separated structural regimes.

We require the target variable to have at least one parent. If the initially sampled graph assigns no parent to the target variable, one to three nodes are randomly selected from those preceding the target in the topological order and connected to the target node. This correction rule guarantees that the target variable is structurally associated with at least one predictive feature.

For each base configuration, the structural function of every non-root node is sampled from a shared pool of random function generators. Function types and their parameters are sampled independently across nodes and across base configurations, but are held fixed across the fourteen experimental conditions associated with the same base configuration. Root nodes have no parents and are generated solely by exogenous random disturbances.

Let $\text{pa}(j)$ denote the parent set of node j . All variables are generated sequentially in topological order. For root nodes, $x_j = \varepsilon_j$; for non-root nodes, generation follows

$$x_j = f_j(x_{\text{pa}(j)}) + \sigma_j \varepsilon_j, \quad \sigma_j = 10^{-s_j/20}, \quad (7)$$

where f_j denotes the sampled structural function, ε_j denotes mutually independent exogenous noise terms, and s_j denotes the SNR of node j in decibels. For every non-root node, the signal term $f_j(x_{\text{pa}(j)})$ and the raw noise term ε_j are each standardized before being combined. Under this scheme, $\sigma_j = 10^{-s_j/20}$ ensures that the SNR actually realized at the node equals the specified s_j exactly. The resulting node value x_j is then standardized again before being passed on to its descendants. Because child nodes receive the noisy x_j rather than the denoised, purely structural signal, perturbations introduced at upstream nodes continue to propagate downward along the graph.

SNR is the second experimental factor and comprises four levels, denoted L1, L2, L3, and Lrand. Given an SNR level, each non-root node independently draws its own SNR value. For L1, L2, and L3, respectively,

$$s_j \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}[-5, 0), \quad s_j \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}[0, 5), \quad s_j \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}[5, 10].$$

Table 13. The fourteen experimental conditions. Parameters specified by $\mathcal{U}[\cdot]$ are independently resampled for each dataset.

Condition	Distribution	Parameters	Characteristic
Normal	$\mathcal{N}(0, 1)$	fixed	symmetric, light-tailed
Laplace	$\text{Laplace}(0, 1)$	fixed	symmetric, heavier-tailed
Uniform	$\mathcal{U}[0, 1]$	fixed	bounded support
Student- t	t_ν	$\nu \sim \mathcal{U}[2.1, 5]$	symmetric, heavy-tailed
Log-normal	$\text{LogNormal}(0, 1)$	fixed	strongly right-skewed
Gumbel	$\text{Gumbel}(0, 1)$	fixed	moderately right-skewed
Exponential	$\text{Exp}(\lambda)$	$\lambda \sim \mathcal{U}[0.5, 2]$	right-skewed
Chi-squared	χ_k^2	$k \sim \mathcal{U}[2.1, 5]$	right-skewed
Beta	$\text{Beta}(\alpha, \beta)$	$\alpha, \beta \sim \mathcal{U}[0.5, 5]$	bounded, flexible shape
Gamma	$\text{Gamma}(k, 1)$	$k \sim \mathcal{U}[0.5, 5]$	right-skewed
Half-normal	$ \mathcal{N}(0, 1) $	fixed	right-skewed
Weibull	$\text{Weibull}(c=1)$	fixed	A/A equivalent to exponential
Mixed	per-node family choice	one of the twelve families per node	heterogeneous across nodes
Clean	no non-root additive noise	$\sigma_j = 0$ for non-root j	reference condition

These three settings confine the node-level SNRs within a graph to narrow intervals of width 5 dB. The three intervals are contiguous and of equal width, jointly forming an equal-width partition of the full range $[-5, 10]$. For Lrand,

$$s_j \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}[-5, 10],$$

so low-SNR and high-SNR nodes can coexist within the same graph. Lrand thereby treats within-graph SNR heterogeneity across nodes as an explicit experimental condition in its own right.

C.2 Noise Diversity and Experimental Settings

We evaluate fourteen experimental conditions in total (see Table 13): twelve single-noise-distribution conditions, one mixed-noise condition, and one reference condition without additive noise. Under each single-distribution condition, all exogenous noise terms within a dataset are drawn from the same distribution family. Under the mixed condition, each node independently selects one of the twelve noise distribution families. Under the Clean reference condition, additive noise is removed from the structural equations of all non-root nodes by setting $\sigma_j = 0$, while root nodes retain the exogenous random variation required to generate non-degenerate data.

All raw noise samples are standardized before entering the structural equations. Consequently, location and scale differences play no role when comparing the single-noise-distribution conditions; differences across conditions arise primarily from the post-standardization distributional shape and, where applicable, from randomly drawn shape parameters. The mixed condition further introduces heterogeneity in the noise distribution family across nodes.

The Exponential and Weibull conditions form a deliberately constructed, distributionally equivalent A/A comparison. A Weibull distribution with shape parameter $c = 1$ is equivalent to an exponential distribution, and the rate parameter of the exponential distribution is eliminated by standardization. The two conditions therefore share the same standardized distribution at the population level, but are generated from mutually independent random samples. The difference between their results serves as an empirical yardstick for the performance fluctuation caused solely by finite-sample randomness, even when the noise laws are exactly equivalent.

The experiment adopts a balanced full factorial design. Graph density has 2 levels and SNR has 4 levels, so crossing them yields 8 experimental combinations. For each combination, we independently sample 20 base configurations. Each base configuration comprises the sampled edge probability and the corresponding graph structure, the function type and function parameters of every non-root node, the node-level SNR vector, and a random permutation of the predictive feature columns. Each base configuration is instantiated once under each of the fourteen experimental conditions, generating a total of $160 \times 14 = 2,240$ datasets.

C.3 Experimental Results

We compare Xiaomi-TabLDM with Limix and TabICLv2. Across all 2,240 datasets, Xiaomi-TabLDM attains the highest R^2 1,575 times, corresponding to a first-place share of 70.3%; by comparison, the first-place shares of LimiX and TabICLv2 are only 16.0% and 13.7%, respectively.

Xiaomi-TabLDM’s relative advantage remains stable across all thirteen non-Clean noise conditions. Its lowest first-place share is still 63.8%, a minimum attained under both the Laplace and Student- t noise conditions, while its highest first-place share reaches 78.1% under the Exponential noise condition. Even under its least favorable noise condition, Xiaomi-TabLDM therefore still ranks first on nearly two-thirds of the datasets. In contrast, LimiX never exceeds a first-place share of roughly 28% under any condition, with its maximum occurring under Student- t noise, while TabICLv2’s highest first-place share is roughly 25%, occurring under the Clean reference condition.

The distributionally equivalent exponential–Weibull pair serves as an internal A/A control. Their first-place shares differ by 5.0 percentage points despite having the same standardized population distribution, indicating that independent sample realizations alone can introduce noticeable variation. This pair therefore provides an empirical reference for the repeatability of the reported condition-wise results.

Under the Clean reference condition, Xiaomi-TabLDM’s first-place share is 65.6%, and under most noisy conditions its first-place share is no lower than this level. This indicates that Xiaomi-TabLDM’s relative competitive advantage does not depend on the absence of additive noise. By contrast, TabICLv2 attains its own highest first-place share under the Clean reference condition, and its relative competitiveness generally declines once additive noise is introduced. Overall, the experimental results show that Xiaomi-TabLDM maintains a stable relative advantage across all tested noise distribution families and perturbation intensities.